
Agen Lightning: Latih Agen AI APAPUN dengan Pembelajaran Penguatan

Xufang Luo*, Yuge Zhang*, Zhiyuan He*, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K.Qiu, Yuqing Yang

Penelitian Microsoft

agen-lightning@microsoft.com
<https://github.com/microsoft/agent-lightning>

Abstrak

Kami menghadirkan Agent Lightning, kerangka kerja yang fleksibel dan dapat diperluas yang memungkinkan pelatihan Model Bahasa Besar (LLM) berbasis Reinforcement Learning (RL) untuk agen AI mana pun. Berbeda dengan metode yang ada yang menggabungkan pelatihan RL dengan agen atau mengandalkannya urutan rangkaian dengan masking, Agen Lightning mencapai pemisahan total di antara keduanya eksekusi dan pelatihan agen, memungkinkan integrasi tanpa batas dengan agen yang sudah ada yang dikembangkan melalui beragam cara (misalnya, menggunakan kerangka kerja seperti LangChain, Agen OpenAI SDK, AutoGen, dan membangun dari awal) dengan modifikasi kode hampir NOL. Oleh merumuskan eksekusi agen sebagai proses keputusan Markov, kami mendefinisikan data terpadu antarmuka dan mengusulkan algoritma RL hierarki, LightningRL, yang berisi kredit modul penugasan, memungkinkan kita menguraikan lintasan yang dihasilkan oleh agen APAPUN memasuki transisi pelatihan. Hal ini memungkinkan RL menangani logika interaksi yang kompleks, seperti skenario multi-agen dan alur kerja dinamis. Untuk desain sistem, kami memperkenalkan a Arsitektur Disagregasi Agen-Pelatihan, dan menghadirkan kerangka kerja observasi agen ke dalam runtime agen, menyediakan antarmuka penyempurnaan agen standar. Eksperimen di seluruh teks-ke-SQL, pembuatan augmented pengambilan, dan tugas penggunaan alat matematika perbaikan yang stabil dan berkelanjutan, yang menunjukkan potensi kerangka kerja ini di dunia nyata pelatihan dan penempatan agen.

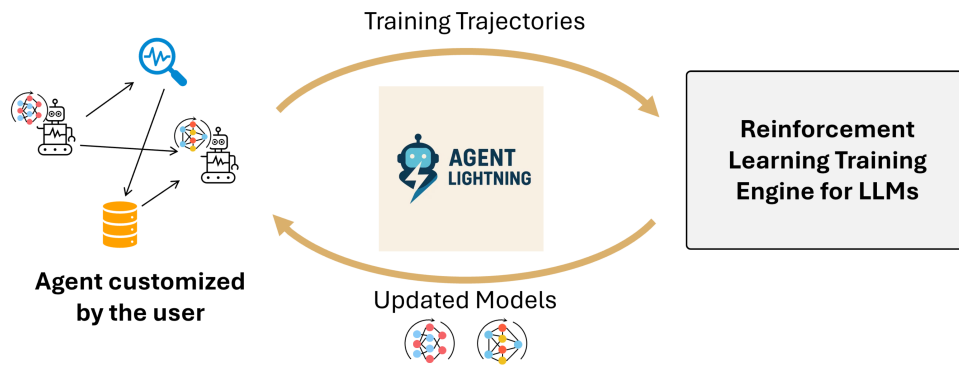
1 Pendahuluan

Kemajuan terkini dalam model bahasa besar (LLM) telah menjadikan agen AI efektif untuk tugas-tugas kompleks seperti pencarian, pembuatan kode, dan penggunaan alat. Agen-agen ini menunjukkan fleksibilitas luar biasa dalam memanfaatkan Kemampuan LLM untuk beradaptasi dengan beragam kebutuhan tugas. Meskipun rekayasa yang cepat dapat membantu meningkatkan kinerja, LLM masih menghadapi keterbatasan yang signifikan. Mereka rentan terhadap kesalahan, terutama saat diterapkan dalam skenario yang tidak dilatih secara eksplisit—seperti alur kerja coding multi-turn, domain pribadi kumpulan data, atau alat yang asing. Akibatnya, agen AI masih kesulitan memecahkan masalah dunia nyata yang kompleks dengan andal tugas-tugas seperti pengembangan perangkat lunak end-to-end dalam produk (Liu et al., 2024). Keterbatasan ini menggarisbawahi perlu melatih atau menyempurnakan model agen untuk sepenuhnya menyadari potensi LLM dalam konteks ini (Chen dkk., 2023; Jin dkk., 2025; Lagu dkk., 2025; Maloo, 2025).

Selain itu, model pelatihan pada agen di dunia nyata kemungkinan akan menjadi sangat penting untuk mendorong kemajuan kemampuan model. Selama eksekusi agen, sistem menghasilkan data interaksi kaya yang menangkap kompleksitas pemecahan masalah di dunia nyata. Pengalaman dunia nyata ini melampaui kumpulan data tradisional yang dikurasi manusia baik dalam skala maupun keragaman, menjadikannya kunci untuk pelatihan LLM di masa depan (Silver & Sutton, 2025; Yao, 2025; MoonshotAI, 2025). Oleh karena itu, memanfaatkan data ini untuk menyempurnakan tidak hanya menyempurnakan keterampilan khusus agen tetapi juga mendorong pengembangan LLM yang lebih serbaguna yang cocok untuk dinamis, lingkungan interaktif.

Pembelajaran penguatan (RL), yang telah mendorong kemajuan terkini dalam model penalaran seperti DeepSeekR1 (Guo et al., 2025) dan Kimi k1.5 (Team et al., 2025), menawarkan paradigma yang kuat untuk mengoptimalkan LLM dalam skenario agen. Meskipun pembelajaran yang diawasi memerlukan anotasi langkah demi langkah yang terperinci—yang mana langka dan mahal untuk tugas interaktif yang kompleks—RL bergantung pada sinyal imbalan berbasis hasil. Ini menghilangkan kebutuhan akan data kurasi khusus tugas dan memungkinkan agen mempelajari perilaku yang diinginkan secara langsung dari umpan balik lingkungan di berbagai tugas. Apalagi sifat trial-and-error dari RL erat mencerminkan bagaimana manusia memperoleh keterampilan pemecahan masalah, memungkinkan model mempelajari kebijakan tindakan secara mendasar dalam konteks penerapan. Kemampuan ini membuka potensi transformasi teks yang dihasilkan LLM token ke dalam tindakan dunia nyata, menjadikan RL cocok untuk model pelatihan dalam sistem berbasis agen.

*Penulis pertama.



Gambar 1: Ikhtisar Agen Lightning, kerangka kerja yang fleksibel dan dapat diperluas yang memungkinkan penguatan pembelajaran LLM untuk agen AI APAPUN.

Namun, memperluas paradigma RL ke agen menimbulkan tantangan besar pada kedua algoritma desain dan implementasi sistem. Metode dan kerangka kerja RL yang ada untuk LLM adalah yang utama disesuaikan untuk tugas panggilan tunggal yang statis, seperti penyalarsan preferensi atau penalaran matematis. Sebaliknya, agen menunjukkan kompleksitas dan keragaman di luar pengaturan ini. Kompleksitasnya terletak pada kenyataan itu eksekusinya sering kali melibatkan beberapa pemanggilan LLM dengan perintah dan respons yang berbeda juga sebagai interaksi dengan alat eksternal, API, atau lingkungan. Keberagaman berasal dari kebutuhan akan desain agen yang berbeda disesuaikan dengan kebutuhan berbagai aplikasi. Tantangan-tantangan ini menghambat penerapan RL untuk penyetelan LLM skala besar di agen.

Kami memperkenalkan Agent Lightning, kerangka kerja yang fleksibel dan dapat diperluas yang memungkinkan pelatihan berbasis RL LLM untuk agen AI APAPUN. Seperti yang ditunjukkan pada Gambar 1, Agen Lightning mencapai pemisahan lengkap antar agen eksekusi dan pelatihan RL, memberdayakan pengembang untuk melatih agen yang ada dengan hampir nol perubahan kode.

Pemisahan ini didasarkan pada perumusan eksekusi agen sebagai proses keputusan Markov (MDP), di mana keadaan mewakili snapshot saat ini dari eksekusi agen, yang terdiri dari nilai-nilai variabel yang memadai menggambarkan status eksekusi. Tindakan tersebut sesuai dengan keluaran yang dihasilkan oleh kebijakan LLM, yaitu kemudian digunakan untuk memperbarui status. Berdasarkan formulasi ini, kami mengusulkan antarmuka data terpadu untuk Pelatihan RL di mana lintasan agen disusun sebagai rangkaian transisi. Setiap transisi berisi keadaan saat ini (masukan LLM), tindakan (keluaran LLM), dan imbalan. Abstrak antarmuka data terpadu ini menghilangkan logika orkestrasi yang mendasari dan detail kerangka agen, sehingga dapat diterapkan pada APAPUN agen. Selanjutnya, untuk mengoptimalkan kebijakan LLM menggunakan transisi yang dikumpulkan, kami memperkenalkan LightningRL, a algoritma RL hierarki yang dirancang khusus untuk pelatihan agen. Setelah menetapkan kredit untuk setiap transisi, LightningRL memperbarui kebijakan dengan cara yang sepenuhnya kompatibel dengan metode RL satu putaran yang ada untuk LLM. Pemodelan dan algoritma tersebut menawarkan banyak manfaat, termasuk memungkinkan konteks yang sangat fleksibel konstruksi, memungkinkan optimalisasi selektif beberapa agen secara bersamaan, dan menghilangkan masalah konteks akumulatif yang mengarah ke urutan yang terlalu panjang.

Berdasarkan formulasi konkrit yang dijelaskan di atas, Agen Lightning memperkenalkan Agen Pelatihan Arsitektur Disagregasi (TA Disagregasi) untuk menetapkan layanan pelatihan standar yang berlaku kepada agen mana pun. Kerangka kerja ini terdiri dari Lightning Server dan Lightning Client. Server Petir berfungsi sebagai pengontrol sistem pelatihan RL, mengelola proses pelatihan dan memaparkan API mirip OpenAI dari model yang diperbarui ke klien. Klien Lightning terdiri dari dua fungsi komponen: satu menangani komunikasi dengan server untuk transmisi dan penerimaan data; yang lain menjalankan agen dan melakukan pengumpulan data, berfungsi sebagai runtime agen.

Memanfaatkan antarmuka data terpadu, runtime agen memungkinkan pemanfaatan yang komprehensif kerangka observabilitas seperti OpenTelemetry (OpenTelemetry, 2025) ke dalam proses pelatihan untuk lintasan koleksi. Hal ini menghubungkan infrastruktur observabilitas dengan pelatihan RL, sehingga memungkinkan teknik pengoptimalan untuk menggunakan data pemantauan yang kaya, dan menciptakan landasan yang meningkatkan skalabilitas dan fleksibilitas. Agen runtime juga dapat memfasilitasi algoritme untuk mengurangi ketersebaran imbalan, melalui Perantara Otomatis Mekanisme Reward (AIR). AIR memungkinkan penetapan imbalan perantara berdasarkan transisi pada sinyal pemantauan sistem (seperti status pengembalian panggilan alat). Pengembang dapat dengan mudah menyesuaikannya mekanisme, yang secara efektif memitigasi masalah imbalan yang jarang pada agen, memungkinkannya menjadi lebih efektif pelatihan.

Kami mendemonstrasikan efektivitas Agen Lightning melalui berbagai tugas, termasuk agen text-to-SQL diimplementasikan dengan LangChain, agen generasi augmented pengambilan yang dibangun dengan OpenAI Agents SDK, dan agen penjawab pertanyaan matematika dengan penggunaan alat yang dikembangkan melalui AutoGen. Kurva imbalannya percobaan menunjukkan bahwa Agen Lightning memungkinkan peningkatan kinerja yang berkelanjutan dan stabil

di berbagai skenario agen, menunjukkan potensi besar untuk menyelesaikan tugas-tugas dunia nyata.

Singkatnya, Agen Lightning memajukan pelatihan agen dalam aspek berikut.

- Agen Lightning adalah kerangka kerja pertama yang mencapai pemisahan penuh antara agen dan pelatihan RL. Pemisahan ini memungkinkan Agen Lightning diterapkan dengan lancar ke agen AI APAPUN pendekatan implementasi, dengan hampir NOL modifikasi kode. Dengan menyelaraskan pelatihan dengan logika eksekusi agen, secara langsung meningkatkan kinerja agen dalam aplikasi dunia nyata. Hal ini memberdayakan pengembang untuk melampaui model statis dan terlatih serta membuka potensi penuh agen adaptif dan berbasis pembelajaran. Ini juga bertindak sebagai antarmuka data terpadu, mengumpulkan beragam data data yang dihasilkan agen untuk meningkatkan kemampuan model. Untuk mencapai pemisahan ini, Agen Lightning memperkenalkan desain khusus dalam algoritma dan infrastruktur.
- Untuk algoritme, Agen Lightning mengembangkan formulasi agen MDP dan memperkenalkan antarmuka data terpadu. Antarmuka ini mengabstraksi kompleksitas berbagai eksekusi agen logika, memungkinkan data yang dikumpulkan selama eksekusi agen untuk langsung diubah menjadi pelatihan lintasan. Selain itu, Agen Lightning menggunakan kerangka RL hierarki dengan kredit modul penugasan yang mengalokasikan pengembalian tingkat lintasan ke respons yang dihasilkan oleh setiap panggilan. Desain ini terintegrasi secara mulus dengan algoritme RL putaran tunggal yang ada, memungkinkan efisiensi dan pelatihan yang efektif.
- Untuk sistemnya, Agen Lightning memperkenalkan arsitektur Disagregasi Agen Pelatihan yang mencapai pemisahan yang jelas antara pelatihan RL dan eksekusi agen. Arsitektur ini terwujud melalui Lightning Server dan Lightning Client, yang bersama-sama menyediakan model standar layanan pelatihan yang berlaku untuk agen mana pun. Klien Lightning berfungsi sebagai runtime agen, secara transparan mengelola eksekusi agen dan mengumpulkan lintasan tanpa memerlukan modifikasi kode. Desain ini memungkinkan penggunaan kembali infrastruktur observasi dalam skenario pelatihan, memastikan ekstensibilitas, skalabilitas, dan integrasi tanpa hambatan dengan beragam kerangka kerja agen.

2
Agen AI Modern

Mendefinisikan agen AI secara tepat merupakan sebuah tantangan karena keragaman dan evolusi mereka yang cepat. Sementara beberapa

penelitian telah berupaya untuk memformalkan dan mengkategorikan agen AI (Significant Gravitas, 2023; Shen et al., 2023; Antropis, 2024; 2025; AI, 2025; Xu & Peng, 2025), tujuan kami bukanlah memperkenalkan definisi baru. Sebaliknya, kami memberikan rumusan abstrak yang berfungsi sebagai pemahaman umum tentang agen AI: agen AI adalah a sistem perangkat lunak yang menggabungkan satu atau lebih panggilan LLM dalam pelaksanaannya. Definisi luas ini mencakup spektrum yang luas dari agen AI yang ada, mulai dari sistem berbasis alur kerja yang mengikuti standar jalur kode (Anthropic, 2024) hingga multi-agen tingkat lanjut yang mampu melakukan perencanaan dinamis, penalaran, dan operasi cakrawala panjang (Anthropic, 2025; AI, 2025; Xu & Peng, 2025).

Di bagian ini, kami menyajikan pengenalan agen AI modern tingkat tinggi, termasuk komponen umum digunakan dalam konstruksi agen, orkestrasi khas komponen ini, dan kerangka kerja agen dengan mudah agen berkembang.

2.1
Komponen

Secara umum komponen bangunan untuk agen dibedakan menjadi dua jenis.

- LLM, atau lebih umum lagi, model dasar (Foundational Model/FM), berfungsi sebagai landasan pemikiran dan pembangkitan inti mesin di dalam agen. Setiap generasi LLM merupakan pemetaan stateless dari input (prompt) ke suatu keluaran (respon). Karena persyaratan komputasinya yang besar, LLM biasanya dijalankan pada server berkinerja tinggi atau platform cloud, dengan agen yang mengaksesnya melalui titik akhir API – baik melalui penyedia komersial (misalnya OpenAI, Google) atau solusi yang dihosting sendiri (misalnya, vLLM (Kwon et al., 2023), SGLang (Tim, 2025)). Seorang agen dapat memanfaatkan beberapa LLM, masing-masing diidentifikasi oleh URL titik akhir yang unik, untuk melakukan tugas yang berbeda atau mencapai beragam fungsi. Kami mendefinisikan himpunan LLM yang digunakan oleh agen sebagai $M = \{M_i\}_{i=1}^m$, di mana m adalah jumlah LLM di agen. Setiap M_i dapat memiliki model atau versi yang berbeda, dan kami dapat menautkan nama model ke model tersebut parameter θ_i untuk kejelasan.
- Alat adalah fungsionalitas yang dipanggil agen untuk melakukan tugas tertentu, seperti mengambil informasi dari database, mengeksekusi kode, atau berinteraksi dengan API. Alat bisa tidak memiliki kewarganegaraan atau stateful, bergantung pada apakah mereka mempertahankan konteks di seluruh pemanggilan. Selain itu, alat dapat diimplementasikan dengan berbagai cara, termasuk API eksternal, program lokal, atau perpustakaan. Di tren terkini, alat sering kali mengikuti MCP (Model Context Protocol) (Protokol, 2025) atau lainnya protokol komunikasi agen untuk menstandarkan interaksi antara agen dan alat, memungkinkan

integrasi dan interoperabilitas yang mulus. Misalkan $T = \{T_j\}_{j=1}^t$ menunjukkan seperangkat alat yang tersedia untuk agen, dimana t adalah jumlah alat.

2.2 Orkestrasi

Menggunakan kumpulan komponen $M \cup T$ sebagai blok penyusun fungsional, orkestrasi – termasuk eksekusi aliran, ketergantungan, dan urutan antar komponen – biasanya disesuaikan oleh pengguna berdasarkan tugas persyaratan. Contoh detail agen dan alur eksekusinya disajikan di Bagian 3.1.3.

Yang penting, orkestrasi ini sering kali tidak bersifat tetap atau deterministik. Agen AI modern menunjukkan dinamika perilaku di mana LLM dapat menentukan tindakan selanjutnya atau memilih alat berdasarkan konteks yang berkembang. Misalnya, dengan membiarkan LLM memutuskan untuk menyaring pertanyaan atau langsung menjawab pertanyaan, maka Agen RAG di atas dapat memiliki jumlah putaran interaksi yang bervariasi dengan database. Contoh lain adalah agen bermain game. Proses berulang dalam menghasilkan tindakan dan menerima umpan balik dari Lingkungan direpresentasikan sebagai urutan panggilan LLM, di mana setiap panggilan berhubungan dengan pengambilan keputusan langkah dalam permainan. Dengan membuka proses berulang ini, kita dapat menangkap perilaku dinamis agen, dimana LLM dapat menyesuaikan strateginya berdasarkan keadaan permainan saat ini dan tindakan sebelumnya.

Meskipun interaksi dinamis ini menangkap perilaku spesifik tugas yang kaya yang dapat meningkatkan pengoptimalan, hal ini tetap terjadi juga menimbulkan tantangan besar bagi pemodelan data dan pembelajaran hilir, sehingga tidak praktis dan tidak praktis tidak dapat diskalakan untuk mengimplementasikan agen yang berbeda dengan kerangka pelatihan RL saat ini.

2.3 Implementasi

Pengembang memiliki berbagai pendekatan untuk menciptakan agen AI. Beberapa mungkin memilih untuk membangun agen seluruhnya dari awal untuk kontrol dan penyesuaian penuh. Beberapa pengembang lain mungkin memanfaatkan yang sudah mapan kerangka pengembangan agen yang menyediakan infrastruktur dan mempercepat pengembangan. Kerangka kerja ini mencakup platform orkestrasi seperti LangChain (LangChain, 2025), OpenAI Agents SDK (OpenAI, 2025), dan AutoGen (AutoGen, 2025).

Kerangka kerja agen memfasilitasi pengembangan dengan menyediakan komponen modular (LLM dan Alat) dan mekanisme fleksibel untuk orkestrasi dinamis, memungkinkan pengguna menentukan alur kerja yang beradaptasi dengan keduanya persyaratan tugas dan status waktu proses agen. Namun, terlepas dari apakah agen dibuat atau tidak awal atau menggunakan kerangka kerja, mereka biasanya tidak memiliki mekanisme untuk perbaikan diri secara otomatis dan melakukan hal tersebut tidak melibatkan optimasi agen. Dalam skenario dunia nyata, khususnya ketika berhadapan dengan data pribadi, kinerja agen sering kali tidak memenuhi persyaratan, sehingga memerlukan pengoptimalan otomatis.

3 Agen Petir

Kami sekarang mengilustrasikan kerangka kerja Agen Lightning yang diusulkan. Kita mulai dengan mendefinisikan data terpadu antarmuka di Bagian 3.1, yang menentukan data apa yang dikumpulkan selama eksekusi agen, tidak tergantung pada desain agen. Selanjutnya, di Bagian 3.2, kami menunjukkan proses pengambilan keputusan Markov pada agen, berbaring landasan untuk pembelajaran penguatan (RL). Kemudian, di Bagian 3.3, kami menyajikan RL hierarki kami algoritma yang memanfaatkan data yang dikumpulkan di agen untuk memperbarui LLM. Akhirnya, kami menjelaskan penyesuaian kami infrastruktur dengan beberapa elemen desain elegan yang secara signifikan menyederhanakan proses pelatihan agen di Bagian 3.4.1.

3.1 Antarmuka Data Terpadu

Pengoptimalan agen berbasis data pada dasarnya bergantung pada data yang dihasilkan selama eksekusi agen. Oleh karena itu, di Agen Lightning, kami mendefinisikan antarmuka data terpadu yang menentukan bagaimana data ini dimasukkan ke dalam RL algoritma pelatihan. Antarmuka ini sangat umum dan dapat diterapkan pada data yang dikumpulkan dari AI apa pun agen.

Agen AI adalah jenis perangkat lunak khusus, dan seperti semua eksekusi perangkat lunak, dapat direpresentasikan sebagai terarah grafik asiklik (DAG) (Alfred et al., 2007; Abadi et al., 2016). Dalam grafik ini, setiap node berkorespondensi pemanggilan komponen (baik LLM atau alat) dari himpunan $M \cup T$, sedangkan setiap sisi mewakili ketergantungan atau aliran kontrol antar komponen. Namun, mengurai grafik lengkap dari tidak terorganisir jejak eksekusi agen bukanlah hal yang sepele dan, seperti yang telah kami temukan, tidak diperlukan untuk tujuan pelatihan. Oleh Dengan memanfaatkan konsep Markov Decision Processes (MDPs), kita hanya perlu mengidentifikasi keadaan saat ini dan faktor kunci yang memengaruhi transisi status (yaitu panggilan) untuk melakukan pengoptimalan RL.

¹Perhatikan bahwa untuk memastikan referensi yang jelas dan tepat, dalam makalah kami, eksekusi mengacu pada data yang dikumpulkan selama satu proses agen dijalankan. Lintasan/episode menekankan data yang diekstraksi yang digunakan untuk pelatihan RL.

3.1.1 Negara dan Panggilan

Kami mendefinisikan status sebagai cuplikan eksekusi agen, yang mungkin mencakup informasi seperti penghitung program, nilai variabel, tumpukan panggilan, konteks sumber daya, dan banyak lagi. Ini dapat diabstraksi sebagai satu set variabel yang berkembang seiring waktu seiring eksekusi agen. Tugas x yang sama dapat dijalankan beberapa kali, menghasilkan jejak eksekusi yang berbeda karena perilaku dinamis agen. Untuk eksekusi x ke- k , keadaan pada timestep t yang berisi variabel V direpresentasikan sebagai:

$$\text{keadaan}(x, k) = \begin{matrix} \blacklozenge \\ \text{variabel}_{x,k,1} \\ \text{saya} \\ \blacklozenge V \end{matrix} \quad \text{saya}=1 \quad (1)$$

Status berubah terus menerus selama eksekusi agen. Misalnya, penghitung dalam perulangan `for` bertambah, dan manipulasi string sederhana mengubah variabel perantara. Namun, operasi perantara ini dan kode tambahan tidak penting untuk optimasi agen. Di sini, kami terutama berfokus pada variabel-variabel kunci – yang digunakan atau dimodifikasi oleh komponen dan mewakili semantik penting (seperti maksud program) – yang kami sebut Variabel Semantik (Lin et al., 2024). Contoh nyata tentang keadaan dan variabel semantik adalah disediakan di Bagian 3.1.3 dan Gambar 2.

Perubahan Variabel Semantik terjadi melalui Pemanggilan Komponen agen. Asumsikan bahwa k -th eksekusi x terdiri dari N pemanggilan/panggilan komponen:

$$\text{eksekusi}(x, k) = \begin{matrix} \blacklozenge \\ \text{panggilan}_{x,k} \\ \text{saya} \\ \blacklozenge N \end{matrix} \quad \text{saya}=1 \quad (2)$$

di mana pemanggilan komponen ke- i didefinisikan sebagai panggilan:

input_{x,k}
output_{x,k}
metax_{x,k}
panggilan_{x,k}
saya_{x,k}
N

Di sini, $C_i \in M$ UT menunjukkan komponen yang dipanggil (baik LLM atau alat) dalam panggilan ini, $\text{input}_{x,k}$ saya dan $\text{keluaran}_{x,k}$ saya mewakili data masukan dan keluaran, $\text{metax}_{x,k}$ saya berisi informasi meta yang relevan tentang pemanggilan, seperti nama C_i , jenis, versi, titik akhir API, serta parameter runtime seperti strategi pengambilan sampel dan suhu untuk panggilan LLM.

Karena status mewakili cuplikan agen, semua masukan dan keluaran adalah variabel semantik secara spesifik langkah waktu: $\text{masukan}_{x,k}$ saya $\in \text{status1}(x, k)$, $\text{keluaran}_{x,k}$ saya $\in \text{status2}(x, k)$.
(4)
Tidak semua informasi di negara bagian diperlukan atau terlihat oleh komponen yang dipanggil. Misalnya, di Gambar 2, meskipun status berisi 4 variabel semantik, LLM hanya dapat melihat input pengguna kapan menulis kueri pertama dan alat pencarian hanya mengambil kueri yang dihasilkan sebagai masukan.

3.1.2 Hadiah dan Kumpulan Data

Untuk memungkinkan pembelajaran dari data yang dikumpulkan selama eksekusi agen, kami memperkenalkan sinyal reward untuk mengukur kualitas penyelesaian tugas. Setiap eksekusi agen ditambah dengan imbalan skalar $\{r_1, \dots, r_N\}$, dimana masing-masing $r_i \in \mathbb{R}$ sesuai dengan kualitas pemanggilan ke- i . Eksekusi dengan sinyal imbalannya diwakili sebagai:

$$\text{eksekusiR}(x, k) = \begin{matrix} \blacklozenge \\ (\text{panggilan}_{x,k} \\ \text{saya} \\ , r_x, k \\ \text{saya}) \\ \blacklozenge N \end{matrix} \quad \text{saya}=1 \quad (5)$$

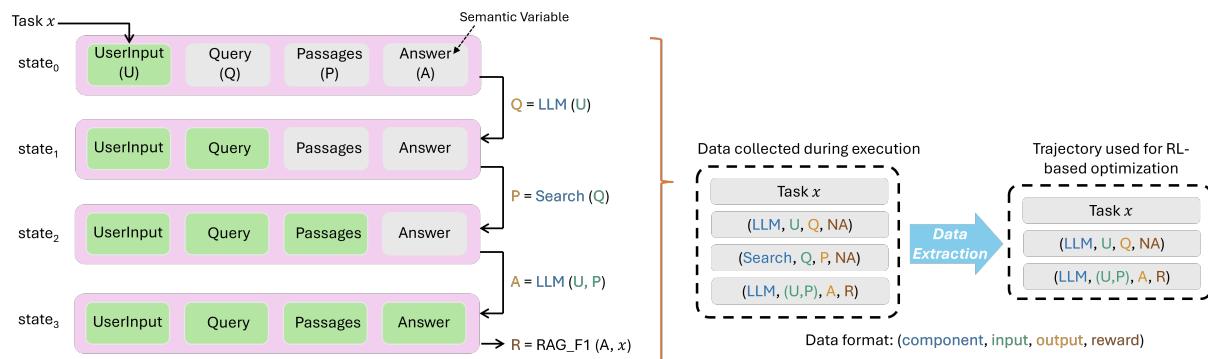
Sinyal imbalan dapat diberikan pada langkah perantara atau pada akhir eksekusi agen, tergantung pada tugas dan desain agen. Imbalan antara r_i ($i < N$), seperti indikator keberhasilan pemanggilan alat atau penyelesaian sebagian tugas, memberikan umpan balik yang lebih terperinci dan memandu pembelajaran agen proses. Hadiah akhir r_N biasanya menilai keberhasilan agen secara keseluruhan dalam menyelesaikan tugas.

Dalam banyak skenario dunia nyata, hanya terminal reward r_N yang tersedia untuk menilai hasil seluruh eksekusi. Ini mewakili kasus khusus dari formulasi umum dimana imbalan menengah $r_1, \dots, r_{N-1}$ tidak ada dan pembelajaran hanya bergantung pada hasil akhir.

Kumpulan data untuk pelatihan atau evaluasi terdiri dari kumpulan tugas $X = \{x_1, \dots, x_{|X|}\}$ dan fungsi hadiah, di mana $|\cdot|$ menunjukkan ukuran yang ditetapkan. Tugas dapat dijalankan beberapa kali, dengan eksekusi diberi keterangan sesuai dengan fungsi hadiah.

3.1.3 Sebuah Contoh Ilustratif

Untuk mengilustrasikan secara konkret eksekusi agen dan antarmuka data terpadu, kami sekarang mempertimbangkan agen generasi retrieveaugmented (RAG), yang ditunjukkan pada Gambar 2. Agen ini dirancang untuk menjawab pertanyaan pengguna dengan pertama-tama mengambil dokumen yang relevan dan kemudian menghasilkan respons berdasarkan dokumen tersebut. Itu



Gambar 2: Ilustrasi antarmuka data terpadu di Agent Lightning. Panel kiri menggambarkan agen aliran eksekusi, di mana setiap transisi keadaan diwakili oleh pembaruan variabel semantik (hijau persegi panjang menunjukkan variabel dengan nilai yang valid; persegi panjang abu-abu menunjukkan variabel yang belum ditetapkan dalam keadaan saat ini). Panel kanan menyajikan lintasan terkait yang dikumpulkan di seluruh agen eksekusi, menunjukkan bagaimana antarmuka data terpadu secara sistematis menangkap semua transisi yang relevan untuk optimasi berbasis RL.

agen terdiri dari dua komponen utama: model kebijakan generatif ($M = \{\text{LLM}\}$) dan alat pencarian ($T = \{\text{Pencarian}\}$).

Alur eksekusi berlangsung sebagai berikut:

1. Pengguna mengirimkan tugas tertentu (x) dengan pertanyaan (UserInput).
2. Agen memanggil LLM untuk menghasilkan permintaan pencarian (Query) yang dikondisikan pada UserInput.
3. Alat Pencarian mengambil bagian yang relevan (Passages) menggunakan Query yang dihasilkan.
4. Agen memanggil LLM lagi untuk menghasilkan jawaban akhir (Answer), memanfaatkan kedua yang diambil Bagian dan UserInput asli.

Setelah selesai, fungsi penghargaan (misalnya RAG F1) mengevaluasi kualitas Jawaban yang dihasilkan. Dengan formulasi kami, setiap negara merangkum nilai-nilai saat ini dari variabel semantik (persegi panjang hijau di Gambar 2 menunjukkan variabel dengan nilai yang valid, sedangkan persegi panjang berwarna abu-abu menunjukkan variabel tanpa nilai dalam keadaan saat ini). Saat agen dijalankan, pemanggilan komponen (LLM atau alat) akan memperbarui komponen yang relevan variabel semantik, menghasilkan perubahan ke keadaan berikutnya. Eksekusi agen untuk tugas tertentu x adalah dengan demikian direpresentasikan sebagai urutan panggilan yang diurutkan (diilustrasikan di sisi kanan Gambar 2), dengan setiap panggilan terdiri dari komponen, masukan, keluaran, dan imbalan terkait (biasanya hanya tersedia untuk panggilan terakhir).

Yang penting, antarmuka data terpadu kami menangkap semua perubahan status, termasuk yang disebabkan oleh non-LLM komponen, memungkinkan metode pengoptimalan yang fleksibel dan sangat dapat disesuaikan. Misalnya saja mendukung optimisasi selektif agen tertentu dalam sistem multi-agen (detail lebih lanjut ada di Bagian 3.2.2 dan eksperimen di Bagian 4.1), serta pengoptimalan di luar penyempurnaan model, seperti prompt penyetulan. Meskipun makalah ini berfokus pada optimasi yang berpusat pada LLM, ekstensibilitas ini menunjukkan hal tersebut penerapan pendekatan kami secara luas. Detail implementasi tambahan dapat ditemukan di GitHub kami repositori2.

Dengan menangkap konteks eksekusi lengkap di setiap negara bagian, antarmuka data terpadu kami menyediakan cukup informasi untuk pembelajaran kebijakan, mencapai pemisahan yang bersih antara eksekusi agen dan pelatihan RL. Desain ini mengakomodasi logika interaksi agen yang arbitrer dan kompleks tanpa memerlukan logika eksplisit penguraian seluruh DAG eksekusi, sehingga sangat menyederhanakan pengoptimalan berbasis RL untuk beragam agen alur kerja.

3.2 Proses Keputusan Markov di Agen

3.2.1 Formulasi

Mari kita mulai dengan kasus sederhana: agen yang berisi satu LLM untuk dioptimalkan. Saat kita menangani LLM ini sebagai model kebijakan, kita dapat memodelkan proses pengambilan keputusannya sebagai Keputusan Markov yang Dapat Diamati Sebagian Proses (POMDP) yang menjadi landasan penerapan RL. Tupel POMDP $M = (S, O, A, P, R)$ dapat didefinisikan sebagai:

²<https://github.com/microsoft/agent-lightning/tree/main/examples/apo>

- S adalah ruang keadaan, berkorespondensi dengan semua keadaan yang mungkin terjadi, yaitu keadaan $\in S$;
- O adalah ruang pengamatan, yang sesuai dengan semua kemungkinan masukan pada LLM kebijakan, yaitu masukan $\in HAI$;
- A adalah ruang tindakan, di mana satu tindakan didefinisikan sebagai seluruh rangkaian token yang dihasilkan dari satu pemanggilan kebijakan LLM, yaitu output $\in A$.
- T (s|s, a) mendefinisikan dinamika transisi (yang biasanya tidak diketahui) ke keadaan baru;
- R(s, a) adalah fungsi imbalan, yang memetakan pasangan tindakan negara ke imbalan skalar;

Konkritnya, pada setiap langkah t , snapshot agen dinyatakan, dan LLM mengamati masukan konteks. Sebagai dijelaskan pada Bagian 3.1.1, input adalah variabel semantik yang mewakili bagian statet yang terlihat atau diperlukan bagi kebijakan LLM untuk membuat keputusan. LLM kemudian menghasilkan serangkaian token output $= (y_t, 1, y_t, 2, \dots, y_t, N_t)$ sebagai outputnya. Urutan ini diperlakukan sebagai satu tindakan, dilambangkan dengan $\in A$. Setelah mengeksekusi, agen bertransisi ke status baru $t+1$. Hadiah skalar $r_t = R(s_t, a_t)$ mungkin disediakan untuk mengevaluasi kualitas tindakan. Pengembaliannya didefinisikan sebagai jumlah imbalan:

$$R = \sum_{t=1}^T r_t$$

Tujuan model kebijakan adalah memaksimalkan keuntungan.

3.2.2 Ekstraksi Data untuk RL

Berdasarkan rumusan MDP di atas, kita perlu mengumpulkan lintasan yang memuat seluruh keputusan kebijakan LLM dan hadiah terkaitnya untuk RL. Untuk tujuan ini, kami mengekstrak dari setiap eksekusi (yaitu Persamaan (5)) hanya informasi yang relevan untuk memperbarui kebijakan LLM yang diparameterisasi oleh θ : input mentah, output LLM panggilan dan imbalannya, dilambangkan sebagai:

$$\begin{aligned} & \text{ekstraksiRL}(x, N) = \\ & \quad \bullet \text{inputmentah}, \\ & \quad \bullet \text{keluaranLLM}, \\ & \quad \bullet \text{imbalan} \\ & \quad \bullet \text{t} \\ & \quad \bullet \text{dengan kegunaan}, \\ & \quad \bullet \text{m} \\ & \quad \bullet \text{m} \end{aligned} \quad t=1, \dots, N \quad (6)$$

Di sini, π_θ adalah LLM yang akan dioptimalkan dan T adalah jumlah pemanggilan dalam lintasan ini. Gambar 2 mendemonstrasikan proses ekstraksi data ini.

Panggilan LLM tidak memiliki kewarganegaraan dan sangat fleksibel. Masukan mereka dapat menyandikan berbagai informasi yang dihasilkan oleh

logika agen yang kompleks, termasuk namun tidak terbatas pada petunjuk instruksi yang menentukan peran LLM atau tujuan, riwayat percakapan, dan konten seperti pertanyaan pengguna, langkah penalaran, dokumen yang diambil bervariasi antar tugas dan eksekusi. Kerangka agen selanjutnya dapat merender atau menguraikan masukan ini dan keluaran (misalnya, melalui templat atau format terstruktur). Untuk tujuan pelatihan RL, terima kasih kepada MDP formulasinya, kita dapat mengabaikan logika dan pemrosesan yang rumit dan bervariasi, dengan hanya berfokus pada input dan output saat ini dari LLM. Pendekatan ini menghilangkan alasan dan sumber rinci untuk memperolehnya masukan LLM. Ekstraksi seperti itu membantu kita menghindari penguraian jejak yang membosankan namun tidak sepele yang dihasilkan oleh dinamika yang sangat tinggi

agen berjalan, sehingga memungkinkan penerapan RL untuk mengoptimalkan agen AI APAPUN.

Singkatnya, antarmuka data terpadu dan formulasi MDP memungkinkan Agen Lightning untuk memisahkan dengan jelas desain agen khusus tugas dari optimalisasi kebijakan berbasis pembelajaran, meletakkan dasar yang efektif Pengoptimalan berbasis RL dalam sistem agen modular dan dinamis.

Aplikasi untuk pengaturan multi-agen LLM tunggal

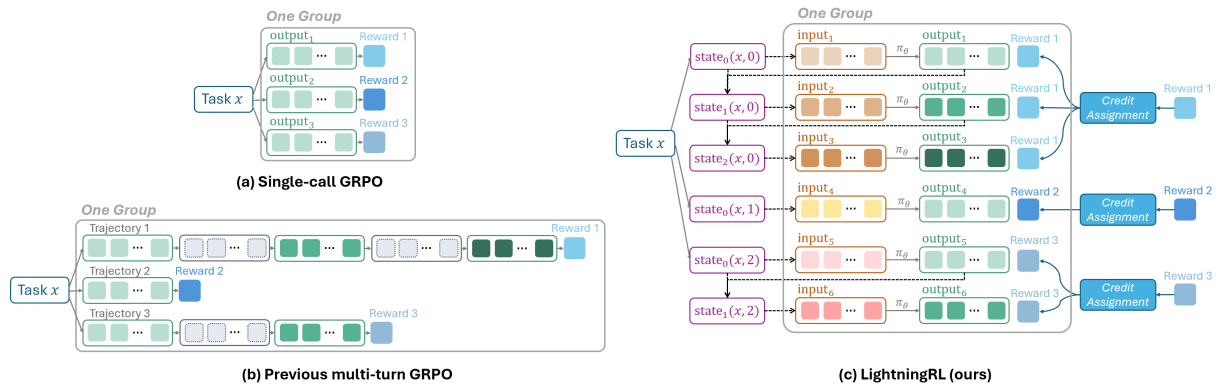
Perhatikan bahwa rumusan di atas berlaku secara fleksibel

skenario multi-agen dengan satu LLM, di mana LLM dapat mengambil banyak peran di berbagai tahapan eksekusi berdasarkan prompt. Dalam kasus RAG pada Gambar 2, selama panggilan LLM pertama, kita dapat memberikan instruksi model untuk fokus pada pencarian, menghasilkan permintaan pencarian a_1 . Kueri ini mengubah status, membentuk status baru masukan. Selama panggilan LLM kedua, kita dapat menginstruksikan model untuk fokus menjawab pertanyaan yang diberikan, menghasilkan jawaban akhir a_2 . Kualitas jawaban ini menentukan imbalan akhir r_2 . Dengan cara ini, a LLM tunggal dapat bertindak sebagai dua agen. Selain itu, Agen Lightning memungkinkan pengoptimalan agen secara selektif di dalamnya sistem multi-agen, dengan menyertakan transisi agen terkait dalam proses pengoptimalan. Dibandingkan dengan pendekatan masking, pemilihan transisi jauh lebih mudah dan intuitif.

Ekstensi ke pengaturan multi-LLM

Ketika beberapa LLM berbeda dengan parameternya sendiri harus ada

diptimalkan bersama, pendekatan langsung memperlakukan setiap LLM sebagai MDP independen dan mengoptimalkannya mereka secara terpisah. Meskipun hal ini menyederhanakan pelatihan, hal ini mengabaikan saling ketergantungan antara kebijakan dan kebijakan menyebabkan koordinasi yang kurang optimal. Pendekatan yang lebih berprinsip akan menggunakan penguatan multi-agen pembelajaran (MARL) atau teori permainan, memperlakukan setiap LLM sebagai agen dengan tujuan dan kebijakannya sendiri (Lowe dkk., 2017; Zhang dkk., 2021). Pendekatan seperti itu lebih mampu menangkap potensi dinamika interaksi dan perilaku kompetitif atau kooperatif di antara beberapa LLM.



Gambar 3: Ilustrasi algoritma LightningRL. (a) GRPO panggilan tunggal, dimana LLM menghasilkan a respons terhadap tugas dalam sekali jalan. Keluaran untuk tugas yang sama dikelompokkan bersama untuk estimasi keuntungan. (b) GRPO multi-putaran sebelumnya. Setiap lintasan berisi beberapa panggilan LLM, dengan lintasan yang sama tugas dikelompokkan untuk estimasi keuntungan. Token yang tidak dihasilkan oleh LLM disamakan (abu-abu putus-putus kotak) selama optimasi. (c) LightningRL yang kami usulkan. Lintasan didekomposisi menjadi transisi, dan transisi untuk tugas yang sama dikelompokkan untuk estimasi keuntungan. Setiap transisi mencakup masukan/konteks, keluaran, dan imbalan saat ini. Masukkannya adalah bagian dari status agen saat ini, dengan imbalan dihitung oleh modul penugasan kredit.

3.3 LightningRL: Metode RL Hierarki untuk Mengoptimalkan LLM di Agen

3.3.1 Pendahuluan Pembelajaran Penguatan Satu Putaran untuk LLM

Kemajuan terbaru dalam RL untuk LLM berfokus pada masalah panggilan tunggal, di mana model tersebut menghasilkan a respons terhadap prompt dalam satu pass. Algoritme ini terutama diterapkan dalam domain seperti matematika dan teka-teki logika (Guo et al., 2025; Xie et al., 2025). Dalam pengaturan ini, jika diberi tugas x , LLM menghasilkan a output respons $= (y_1, \dots, y_N)$ token demi token dari kebijakan π_θ , dan setiap token diperlakukan sebagai tindakan. Setelah seluruh respon dihasilkan, imbalan skalar $r(x, \text{output}, \hat{y})$ diberikan berdasarkan kebenarannya. atau kualitas solusi, dimana $\hat{y}$ adalah kebenaran dasar untuk tugas x . Kerugian tingkat token yang khas bisa terjadi disederhanakan sebagai3:

$$L(\theta) = -\mathbb{E}_{x \sim X} \left[\sum_{j=1}^N \log \pi_\theta(y_j | x, y_{<j}) \cdot A_j \right] \quad (8)$$

di mana A_j adalah perkiraan keuntungan tingkat token, dan X adalah kumpulan tugas. Perbedaan inti antara populer algoritma terletak pada estimasi istilah keuntungan, A . Standar PPO (Ouyang et al., 2022) mempekerjakan a model kritik yang dipelajari, mendefinisikan keunggulan dengan fungsi nilai. Sebaliknya, metode terkini menghilangkan perlunya fungsi nilai terpisah untuk menyederhanakan pelatihan. Misalnya, GRPO (Shao et al., 2024) menghitung keuntungan dengan menormalkan imbalan suatu respons terhadap mean dan deviasi standar imbalan dari sekelompok respons yang dihasilkan untuk prompt yang sama. Demikian pula, PERKUAT++ (Hu, 2025) menggunakan garis dasar yang lebih sederhana, yang mendefinisikan keuntungan sebagai perbedaan antara imbalan yang diberikan oleh respons saat ini dan hadiah rata-rata dari seluruh kelompok pelatihan.

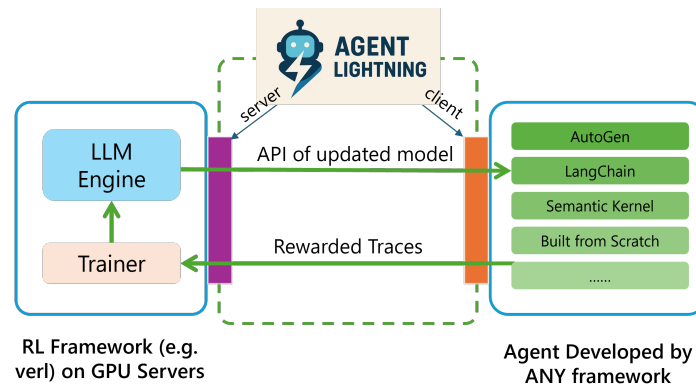
3.3.2 Perluas ke Skenario Agen melalui LightningRL

Dalam formulasi MDP di atas, kami memperlakukan seluruh rangkaian token yang dihasilkan selama satu pemanggilan LLM sebagai satu tindakan. Kebijakan tersebut mungkin perlu mengambil beberapa tindakan melalui interaksi agar dapat diselesaikan satu episode penuh. Sebaliknya, LLM menghasilkan keluaran token demi token, dan sebagian besar algoritma RL yang ada menghasilkan output token demi token

LLM dirancang untuk interaksi satu putaran, dengan fokus pada pengoptimalan pembuatan token dalam satu putaran tanggapan. Untuk menjembatani kesenjangan ini, Agen Lightning memperkenalkan pembelajaran penguatan hierarki sederhana (HRL), disebut LightningRL. Pendekatan ini terintegrasi secara mulus dengan RL satu putaran yang ada metode untuk LLM tanpa modifikasi dan secara efektif mendukung pengoptimalan dalam skenario agen apa pun.

Seperti yang ditunjukkan pada Gambar 3, Agen Lightning mengatur data menggunakan transisi, dan setiap transisi didefinisikan sebagai tupel $(\text{input}_x, k, t, r_x, k)$ dalam Persamaan. (6). Setiap transisi berisi konteks masukan untuk transisi saat ini Panggilan LLM, terlepas dari bagaimana konteksnya dibangun. LightningRL menerapkan mekanisme dua langkah: the pengembalian tingkat episode R pertama kali ditetapkan di seluruh tindakan oleh modul penugasan kredit; maka itu lebih jauh

3Perhatikan beberapa perbaikan dan kendala, seperti rasio pengambilan sampel penting, kliping, dan divergensi KL istilah-istilah tersebut dihilangkan dalam persamaan ini, tetapi istilah-istilah tersebut diadopsi sesuai dengan algoritma akhir.



Gambar 4: Arsitektur Disagregasi Agen Pelatihan.

didekomposisi menjadi beberapa token dalam setiap tindakan, menghasilkan sinyal pengawasan tingkat token. Di masa kini kita implementasinya, LightningRL hanya mengasumsikan setiap tindakan dalam episode tersebut memiliki nilai yang sama, setara ke pengembalian akhir R . Langkah kedua kemudian ditangani oleh algoritma RL putaran tunggal yang ada untuk LLM.

Desain ini membawa beberapa keunggulan. Pertama, ini memungkinkan penggunaan langsung algoritma RL putaran tunggal apa pun tanpa modifikasi, terutama metode bebas nilai yang biasanya ringan dan efisien. Misalnya saja di GRPO, sampel yang dihasilkan dari prompt yang sama dikelompokkan untuk memperkirakan keuntungan. Kami menerapkan hal yang sama pendekatan di sini: setiap tugas x dijalankan beberapa kali untuk menghasilkan data eksekusi yang berbeda, masing-masing didekomposisi menjadi tindakan individu. Sampel ini kemudian dikelompokkan berdasarkan tugas untuk menghitung statistik dalam Persamaan. 8. Serupa adaptasi berlaku untuk PPO dan REINFORCE++.

Kedua, LightningRL memungkinkan konstruksi observasi kebijakan yang fleksibel, karena data kami disusun di tingkat transisi individu, masing-masing dengan imbalannya sendiri. Input/pengamatan saat ini dapat dilakukan secara fleksibel berasal dari negara. Misalnya, masukan dapat berupa ringkasan langkah-langkah sebelumnya yang dihasilkan oleh LLM, atau prompt terstruktur yang dikumpulkan melalui templat, atau instruksi yang menunjukkan peran LLM saat ini (dijelaskan di Bagian 3.2.2). Sebaliknya, metode sebelumnya sering kali menggabungkan semua belokan dalam suatu lintasan menjadi satu respons dan menggunakan masking untuk mengontrol bagian mana yang diperbarui – sebuah pendekatan yang tidak bisa dilakukan mendukung konstruksi konteks yang fleksibel dan modular.

Ketiga, dibandingkan dengan strategi masking, LightningRL menawarkan implementasi yang lebih kuat dan terukur. Pendekatan berbasis masking tidak hanya memerlukan penggabungan yang erat antara pelatihan dan logika eksekusi agen, tetapi juga memerlukan hubungan yang erat antara pelatihan dan logika eksekusi agen mengganggu kontinuitas token di LLM, yang diasumsikan dalam pengkodean posisi yang banyak digunakan pendekatan, seperti Rotary Positional Embeddings (RoPE) (Su et al., 2024). Selain itu, penyamaran menimbulkan kompleksitas yang signifikan dalam verifikasi kode, debugging, dan desain kernel, yang sering kali mengakibatkan berkurangnya efisiensi ketika masker menjadi rumit. Sebaliknya, LightningRL menguraikan lintasan agen menjadi transisi, secara alami menyelaraskan dengan struktur masukan LLM dan menghilangkan kebutuhan akan tambahan penyamaran. Desain ini menyederhanakan implementasi dan meningkatkan skalabilitas. Apalagi pengorganisasian data di bentuk transisi meringankan permasalahan yang disebabkan oleh konteks akumulatif dan konteks yang terlalu panjang dihasilkan dari penggabungan semua belokan. Lintasan panjang dipecah menjadi beberapa transisi, memungkinkan penggunaan teknik seperti akumulasi batch untuk pembaruan yang efisien.

Penugasan kredit yang lebih canggih

Modul penugasan kredit memungkinkan integrasi di masa depan

strategi yang lebih canggih, seperti pemberian kredit berdasarkan heuristik atau model yang dipelajari. Satu jalan potensial untuk pekerjaan di masa depan adalah dengan memperkenalkan fungsi nilai tingkat tinggi untuk memperkirakan apa yang diharapkan pengembalian untuk setiap tindakan secara individual, memberikan distribusi kredit yang lebih bernaunsa. Namun demikian, milik kita Hasil eksperimen menunjukkan bahwa strategi penugasan identik sederhana efektif di banyak bidang skenario dan kumpulan data.

Pelatihan LLM untuk agen dunia nyata merupakan upaya menantang yang memerlukan integrasi pelatihan RL kerangka kerja dan kerangka pengembangan agen. Kerangka kerja ini seringkali rumit, berkembang pesat, dan pada dasarnya terfragmentasi, sehingga sulit untuk diselaraskan. Alamat Agent Lightning Framework tantangan ini dengan menawarkan solusi terpadu yang memfasilitasi pengembangan dan pelatihan yang lancar agen menggunakan RL dan teknik optimasi lainnya.

Kerangka pelatihan RL untuk LLM terdiri dari dua komponen utama: pelatih dan peluncuran. Pelatih bertanggung jawab untuk memperbarui bobot model, sementara peluncuran mencatat lintasan yang adalah data yang digunakan oleh pelatih. Untuk tugas RL satu putaran, seperti penalaran matematika, proses peluncuran sangat mudah, hanya melibatkan pembuatan respons LLM berdasarkan petunjuk spesifik. Sebaliknya, untuk tugas agen yang melibatkan banyak putaran dan interaksi dengan lingkungan, peluncuran diperlukan eksekusi lengkap agen. Ini mencakup tidak hanya beberapa contoh generasi LLM tetapi juga eksekusi kode asli, seperti pemanfaatan alat.

Inovasi inti Agen Lightning adalah arsitektur Disagregasi Agen Pelatihan, yang memisahkan generasi LLM yang intensif komputasi dan logika aplikasi yang ringan namun beragam dan fleksibel dan alat yang ditulis dalam bahasa pemrograman tradisional. Yang pertama dikelola oleh kerangka RL, dan mengekspos API mirip OpenAI ke yang terakhir, diilustrasikan oleh Gambar 4. Yang terakhir dapat dikelola dan dijalankan secara mandiri, tidak perlu ditempatkan bersama dengan sumber daya GPU, sehingga memungkinkan fleksibilitas yang lebih besar dalam pendefinisian perilaku agenik.

Pelatihan Agen-Agnostik dan Agen Pelatih-Agnostik
Keuntungan utama dari Agen Lightning

Arsitektur Disagregasi Agen Pelatihan adalah kemandirian timbal balik yang dibangun di antara kerangka pelatihan dan agen. Desain ini menjadikan kerangka pelatihan (misalnya, VeRL (Sheng et al., 2024)) agen-agnostik; satu-satunya perhatiannya adalah optimalisasi LLM dan pengelolaan perangkat keras sumber daya, tanpa digabungkan dengan logika agen tertentu. Sebaliknya, agen, beroperasi pada klien di sisi lain, bersifat agnostik terhadap pelatih, sehingga memungkinkannya berfungsi secara independen dari implementasi kerangka pelatihan detail. Pemisahan ini memberikan fleksibilitas yang signifikan dalam pengembangan dan penerapan agen, memberdayakan pengembang untuk berkonsentrasi pada logika agen tanpa kendala infrastruktur pelatihan. Lampiran A memberikan contoh kode yang menunjukkan bagaimana agen yang ada dapat dioptimalkan dengan mudah menggunakan Agen Lightning, menunjukkan kemudahan penggunaan dan kemampuan beradaptasi.

Untuk memenuhi tujuan desain di atas, Agen Lightning memperkenalkan arsitektur dua komponen: Agen Lightning Server dan Agent Lightning Client, seperti yang digambarkan pada Gambar 4. Server dirancang untuk berintegrasi secara mulus dengan kerangka kerja RL, mengelola proses pelatihan dan optimalisasi LLM. Itu menangani orkestrasi tugas, manajemen data, dan komunikasi dengan klien. Klien merangkul agen, memungkinkan mereka untuk beroperasi secara independen dari kerangka pelatihan, sehingga memungkinkan fleksibilitas dalam pengembangan dan eksekusi agen. Setelah menerima kumpulan data tugas yang diunggah oleh pengguna, server, yang diinisialisasi bersama dengan kerangka RL, mengatur prosedur pelatihan. Server membagi kumpulan data menjadi beberapa batch dan mengelola transisi antara tahap pembuatan dan pelatihan menggunakan sistem berbasis peristiwa, berkoordinasi dengan klien. Kumpulan tugas dikirim ke klien melalui cara inventarisasi, di mana server menyimpan catatan tugas yang tersedia dan menugaskannya ke klien saat mereka siap. Titik akhir API mirip OpenAI, yang unik untuk setiap tugas, diteruskan ke klien bersamaan dengan tugas tersebut. Titik akhir ini memungkinkan klien untuk terhubung ke server dan memfasilitasi proses penelusuran dan pengambilan data. Klien kemudian mengeksekusi agen di runtime agen (Bagian 3.4.2) untuk menghasilkan jejak dan imbalan. Jejak dikumpulkan dan dilaporkan kembali ke server, yang memproses data dan meneruskannya ke kerangka pelatihan untuk pembaruan parameter model. Arsitektur ini memungkinkan pelatihan agen yang efisien dan terukur, memanfaatkan kekuatan kerangka RL dan agen kerangka pembangunan.

Klien Agen Lightning adalah manajer runtime untuk agen yang mengatur eksekusi, menangkap data, menangani kesalahan, dan berkomunikasi dengan server. Ini mendukung alur kerja pelatihan agen menjadi kuat, terukur, dan efisien.

Paralelisme Data untuk Eksekusi Agen

Ukuran batch yang besar sangat penting dalam pelatihan RL modern agar dapat sepenuhnya memanfaatkan sumber daya komputasi dan mengurangi latensi peluncuran, yang merupakan penghambat pelatihan lingkaran. Untuk memproses batch besar ini, beberapa instans agen harus dijalankan secara paralel. Agen Petir klien dirancang untuk mengelola dan mengeksekusi beberapa instance agen secara bersamaan secara efisien, dengan memanfaatkan paralelisme data untuk memaksimalkan throughput dan meminimalkan latensi. Khususnya, ini menerapkan dua tingkat strategi paralelisme: paralelisme intra-node, di mana satu klien menjalankan banyak pekerja pada satu mesin, masing-masing mengeksekusi instance agen yang berbeda, dan paralelisme antar-node, tempat banyak klien berjalan mesin yang berbeda, masing-masing mengelola kumpulan instance agennya sendiri. Arsitektur ini memungkinkan untuk fleksibel penskalaan dan pemanfaatan sumber daya yang efisien di seluruh sistem terdistribusi.

Pengambilan Data tanpa Modifikasi Kode
Untuk berintegrasi secara lancar dengan basis kode agen yang ada, Klien Lightning menggunakan dua teknik instrumentasi yang menangkap data relevan tanpa memerlukannya

modifikasi logika agen. Teknik pertama didasarkan pada OpenTelemetry (OpenTelemetry, 2025) dan AgenOps (AgenOps, 2025). Ia menggunakan kemampuan penelusuran OpenTelemetry untuk melakukan instrumen secara otomatis kode agen, menangkap jejak eksekusi, panggilan LLM, dan interaksi lingkungan. Bagi pengguna yang lebih suka tidak bergantung pada OpenTelemetry, Agent Lightning juga menyediakan mekanisme tracing dasar yang tertanam di dalamnya titik akhir API mirip OpenAI. Mekanisme ini ringan dan terukur, memastikan pengambilan data dapat dilakukan secara efisien bahkan untuk kerangka pengembangan agen yang tidak kompatibel OpenTelemetry.

Penanganan Kesalahan dan Kekokohan

Stabilitas dan keandalan proses pelatihan adalah yang terpenting dan peka terhadap berbagai jenis kegagalan, terutama dalam sesi latihan jangka panjang. Kesalahannya bisa muncul dari berbagai sumber, termasuk kerusakan agen, gangguan jaringan, atau keluaran yang tidak valid. Dan frekuensinya dapat meningkat secara signifikan dalam pelatihan RL dibandingkan dalam inferensi karena sifat bawaan algoritma RL dinamika eksplorasi dan eksploitasi. Kesalahan seperti itu, jika tidak ditangani dengan baik, dapat menimbulkan dampak yang signifikan downtime dan sumber daya komputasi yang terbuang. Untuk mengatasi hal ini, klien Agen Lightning menggabungkan mekanisme penanganan kesalahan komprehensif yang menjamin ketahanan selama pelatihan. Ini mendeteksi kegagalan yang tidak ditangani dengan benar oleh kode agen, seperti crash atau panggilan alat yang lama terhenti, dan memastikan bahwa kegagalan ini tidak mengganggu proses pelatihan secara keseluruhan. Tugas yang gagal dapat dicoba ulang atau ditugaskan kembali contoh agen lainnya, dan log terperinci disimpan untuk tujuan debugging dan pemantauan. Ini desain meminimalkan waktu henti dan memaksimalkan efisiensi pelatihan, memungkinkan peningkatan berkelanjutan kinerja agen.

Penghargaan Menengah Otomatis (AIR)

Sifat imbalan yang tertunda dan jarang dalam pelatihan RL dapat menghambat proses pembelajaran. Imbalan menengah, yang memberikan umpan balik pada berbagai tahap eksekusi agen, dapat secara signifikan meningkatkan proses pelatihan dengan menawarkan lebih sering dan informatif sinyal. Namun hal ini sering kali memerlukan overhead yang tinggi seperti anotasi manusia yang mahal atau imbalan yang rumit logika komputasi. Cara mendapatkan imbalan perantara dari eksekusi agen sangatlah penting proses pelatihan. Klien Agen Lightning menyediakan mekanisme AIR untuk mengubah pemantauan sistem data menjadi imbalan menengah, memungkinkan kerangka pelatihan untuk memanfaatkan sinyal-sinyal ini secara lebih baik pembelajaran yang efektif.

Layanan Lingkungan dan Penghargaan untuk Skalabilitas

Lingkungan dan fungsi penghargaan sangat penting komponen pelatihan RL. Untuk lingkungan dan fungsi penghargaan yang ringan dan dapat dijalankan secara lokal, mereka dapat dieksekusi langsung dalam pekerja yang sama dengan instance agen. Tapi untuk yang membutuhkan banyak sumber daya atau dengan biaya inisialisasi yang lebih tinggi, seperti emulator ponsel atau logika komputasi imbalan yang kompleks, akan lebih efisien untuk menghostingnya sebagai layanan bersama. Saat ini, Klien Agen Lightning mendukung layanan hosting untuk lingkungan dan perhitungan penghargaan secara sederhana cara pengumpulan, dan dapat diperluas untuk mendukung arsitektur tanpa server yang lebih kompleks di masa depan.

4
Hasil

Efektivitas Agen Lightning divalidasi pada tiga tugas berbeda, masing-masing diterapkan dengan tugas berbeda kerangka agen. Rangkuman contoh tugas terdapat pada Tabel 1. Seperti yang ditunjukkan hasil berikut ini subbagian, Agen Lightning memungkinkan peningkatan kinerja yang berkelanjutan dan stabil di berbagai bidang skenario, menunjukkan potensi kuat untuk optimalisasi agen di dunia nyata.

Tabel 1: Ringkasan tugas dan pengaturan dalam eksperimen.

Tugas	
Teks-ke-SQL	
Revisi QA	
QA Matematika	
Konfigurasi	
LangChain	
SQL Agent OpenAI	
General Chatbot	
Konfigurasi data	
Local file	
MySQL	
Kode SQL	
API yang digunakan	
PostgreSQL	
Pengontrol Wipeout	
Kalimat	
Monitor agen	
1	
1	
Monitor agen yang distil	
2	
1	

4.1
Teks-ke-SQL melalui LangChain

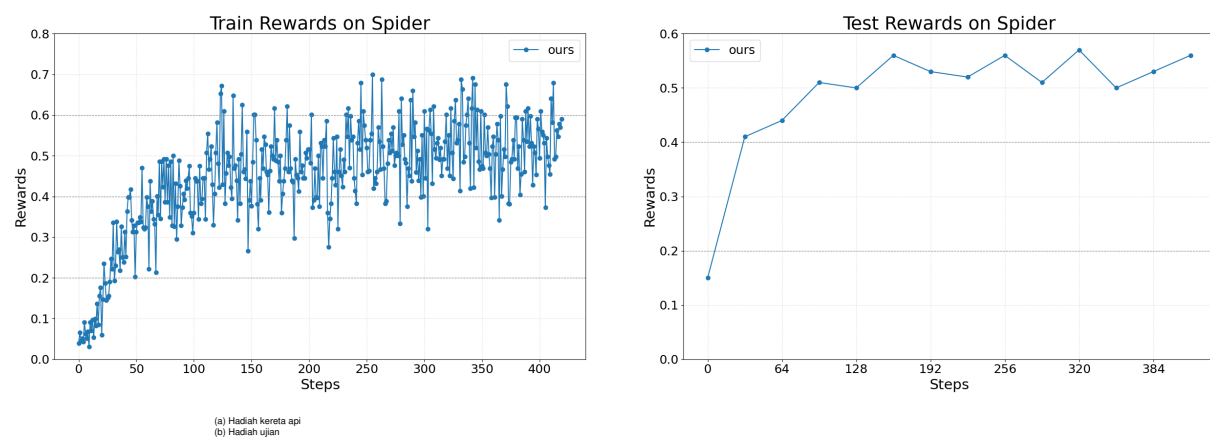
Dalam tugas ini, dengan pertanyaan bahasa alami dan database, agen harus membuat kueri SQL untuk itu mengambil informasi yang relevan, dan kemudian menjawab pertanyaannya. Kami menggunakan dataset Spider (Yu et al., 2018), a tolok ukur text-to-SQL yang kompleks dan lintas domain yang dirancang untuk mengevaluasi kemampuan model dalam menghasilkan kueri SQL yang dapat dieksekusi dari pertanyaan bahasa alami. Ini berisi lebih dari 10.000 pertanyaan di 200

database dengan skema beragam yang mencakup 138 domain berbeda, memerlukan model untuk digeneralisasi hingga tidak terlihat database pada waktu ujian. Kami menggunakan Llama-3.2-3B-Instruct (Meta, 2024) sebagai model dasar.

Kasus ini diimplementasikan menggunakan LangChain, dan dirancang sebagai sistem multi-agen. Ini berisi 3 agen dan alur kerjanya adalah sebagai berikut. Agen penulis SQL pertama-tama membuat kueri SQL, yang kemudian dieksekusi. Jika kuerinya benar, pelaksana SQL mengembalikan informasi dari database; jika salah, itu mengembalikan pesan kesalahan. Selanjutnya, agen pemeriksa mengevaluasi kebenaran kueri SQL dan validitas dan kecukupan informasi yang diambil. Agen ini memutuskan apakah akan menulis ulang kueri atau langsung menghasilkan jawaban. Jika penulisan ulang diperlukan, agen penulisan ulang merevisi kueri berdasarkan umpan balik pemeriksa LLM. Jika tidak diperlukan penulisan ulang, agen penulisan ulang juga bertanggung jawab untuk menjawab pertanyaan, mengambil informasi yang diambil dan pertanyaan untuk menghasilkan jawaban akhir. Dalam alur kerja ini, pembuatan, pemeriksaan, dan penulisan ulang SQL dilakukan oleh LLM yang sama, namun dengan petunjuk berbeda yang disesuaikan dengan setiap tugas, secara efektif berfungsi sebagai 3 agen. Dan selama pelatihan, kami saja optimalkan dua di antaranya, yaitu agen penulisan SQL dan penulisan ulang. Dan kedua agen ini dioptimalkan secara bersamaan. Kasus ini menunjukkan bahwa Agen Lightning dapat mengoptimalkan satu atau lebih agen secara selektif dalam sistem multi-agen.

Tugas ini mengharuskan LLM untuk memiliki berbagai kemampuan, termasuk menghasilkan dan mengoreksi kueri SQL yang valid secara sintaksis dan semantik, memeriksa kualitas kueri dan informasi yang diambil, memahami konteks saat ini untuk menilai apakah informasi yang diperoleh sudah cukup, meningkatkannya SQL untuk mendapatkan informasi yang benar, dan akhirnya menjawab pertanyaan berdasarkan yang diambil hasil.

Hadiah untuk tugas ini ditentukan oleh benar atau tidaknya jawaban akhir atas pertanyaan tersebut. Modelnya Kinerja akhir juga dievaluasi berdasarkan keakuratan jawaban pada set tes.



Gambar 5: Kurva imbalan untuk tugas Text-to-SQL.

Seperti yang ditunjukkan pada Gambar 5, Agen Lightning memungkinkan peningkatan hadiah yang stabil, menunjukkan kemampuannya mengoptimalkan keputusan multi-langkah kompleks yang melibatkan pembuatan kode dan penggunaan alat.

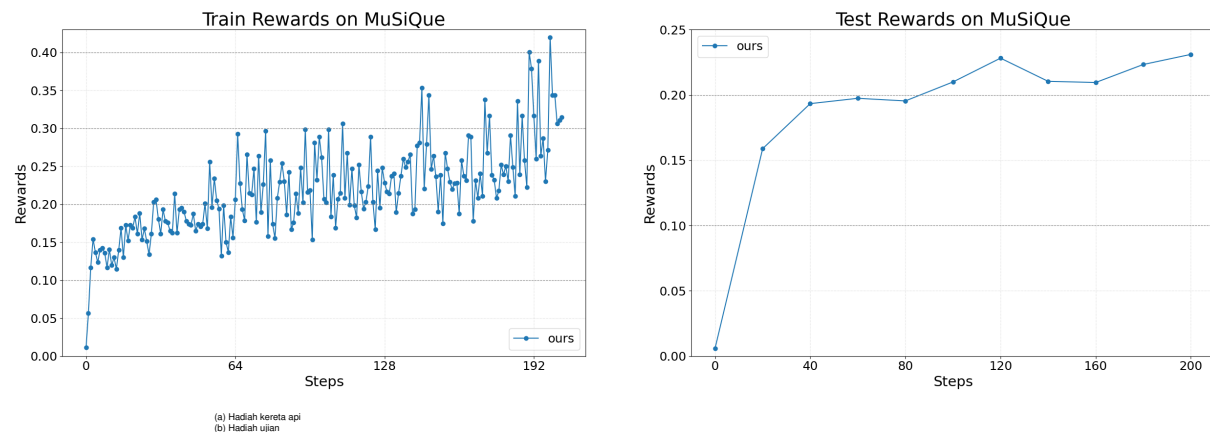
4.2 Generasi Pengambilan Augmented melalui OpenAI Agents SDK

Yang kedua adalah tugas khas Retrieval-Augmented Generation (RAG). Diberikan pertanyaan dan a database dokumen, agen pertama-tama membuat kueri bahasa alami untuk mengambil dokumen pendukung menggunakan alat retriever yang ada. Kami menggunakan kumpulan data MuSiQue (Trivedi et al., 2022), sebuah tolok ukur jawaban pertanyaan multihop yang menantang yang dirancang untuk mempromosikan penalaran komposisional yang asli. Berbeda dengan tolok ukur sebelumnya yang seringkali dapat diselesaikan dengan menggunakan jalan pintas yang dangkal, MuSiQue dibangun menggunakan a metodologi bottom-up yang secara sistematis menyusun pertanyaan-pertanyaan satu lompatan yang terhubung, memastikan hal itu langkah penalaran sangat bergantung pada keluaran langkah penalaran lainnya. Selain itu, untuk meniru pengaturan pencarian di dunia nyata, database yang diberikan adalah seluruh Wikipedia, berisi 21 juta dokumen. Dan kami menggunakan embeddings dihasilkan oleh model BGE (BAAI, 2023; Zhang et al., 2023) dan kesamaan kosinus sebagai alat retriever kami. Pertanyaan multi-hop dan sumber pencarian yang besar ini menunjukkan tantangan besar bagi agen. Kami menggunakan Llama-3.2-3B-Instruct (Meta, 2024) sebagai model dasar.

Agen ini diimplementasikan menggunakan OpenAI Agents SDK. Alur kerja agen di sini mirip dengan tetapi lebih sederhana dari tugas text-to-SQL sebelumnya. Kebijakan LLM harus terlebih dahulu menghasilkan kueri, dan kemudian memutuskan apakah akan menyaring kueri atau menghasilkan jawaban berdasarkan dokumen yang diambil. Satu LLM digunakan dalam alur kerja ini dan kami tidak menggunakan pengaturan multi-agen dalam kasus ini, artinya hanya itu satu instruksi yang berisi semua hal yang perlu dilakukan oleh LLM digunakan. Berbeda dengan tugas SQL, tugas

pertanyaan di sini adalah teks bebas, membuat langkah pengambilan dan penalaran lebih terbuka. Juga, yang diberikan database jauh lebih besar dari tugas sebelumnya. Tugas ini menguji kemampuan agen untuk merumuskan secara efektif kueri pengambilan dengan makna semantik dan alasan atas informasi tekstual yang diambil.

Hadiah yang digunakan selama pelatihan adalah kombinasi tertimbang dari skor format Rformat dan skor kebenaran. Secara khusus, skor format adalah 1 jika LLM dapat menghasilkan output dalam format tertentu, dengan konten yang sesuai harus diapit oleh token tertentu, seperti `<think>...</think>`, `<query>...</query>` dan `<jawaban>...</jawaban>`. Dan skor kebenaran Rkebenaran dihitung sebagai skor F1 tingkat kata antara jawaban prediksi dan jawaban emas. Dua skor digabungkan sebagai $R = 0,9 \times R_{kebenaran} + 0,1 \times R_{format}$.



Gambar 6: Kurva imbalan untuk tugas Retrieval-Augmented Generation.

Gambar 6 menunjukkan bahwa Agen Lightning memungkinkan peningkatan kinerja yang stabil dalam tantangan ini tugas, menunjukkan efektivitasnya dalam skenario RAG yang lebih kompleks dan terbuka.

4.3 QA Matematika dengan Penggunaan Alat melalui AutoGen

Tugas ketiga adalah tugas QA yang berfokus pada matematika yang mengevaluasi kemampuan agen untuk memanggil alat (khususnya, kalkulator) untuk memecahkan masalah aritmatika dan simbolik. Kami menggunakan kumpulan data Calc-X (Kadl'c'ik et al., 2023), yang terdiri dari beragam soal matematika yang memerlukan penalaran dan perhitungan yang tepat. Oleh memodifikasi kumpulan data matematika yang ada, seperti GSM8K dan Ape210K, Calc-X menekankan integrasi alat eksternal ke dalam alur kerja penalaran, sehingga cocok untuk menguji agen yang dilengkapi alat. Kami menggunakan Llama-3.2-3B-Instruct (Meta, 2024) sebagai model dasar.

Agen diimplementasikan melalui AutoGen dan mengikuti alur kerja yang sederhana namun intensif alat. Diberikan secara alami pertanyaan matematika bahasa, agen harus memutuskan bagaimana dan kapan menggunakan alat kalkulator untuk menghitung nilai antara sebelum menghasilkan jawaban akhir. Alur kerja dijalankan oleh satu LLM, yang bertanggung jawab untuk menghasilkan panggilan alat, menafsirkan keluaran alat, dan membentuk jawaban akhir. Ini mengharuskan model untuk memahami struktur masalah matematika, mengeluarkan panggilan alat yang benar secara sintaksis, dan mengintegrasikan keluaran alat dengan benar ke dalam rantai penalaran akhir.

Hadiah akhir didasarkan pada apakah agen menjawab pertanyaan dengan benar, dan performa model juga dievaluasi melalui akurasi jawaban pada set pengujian.

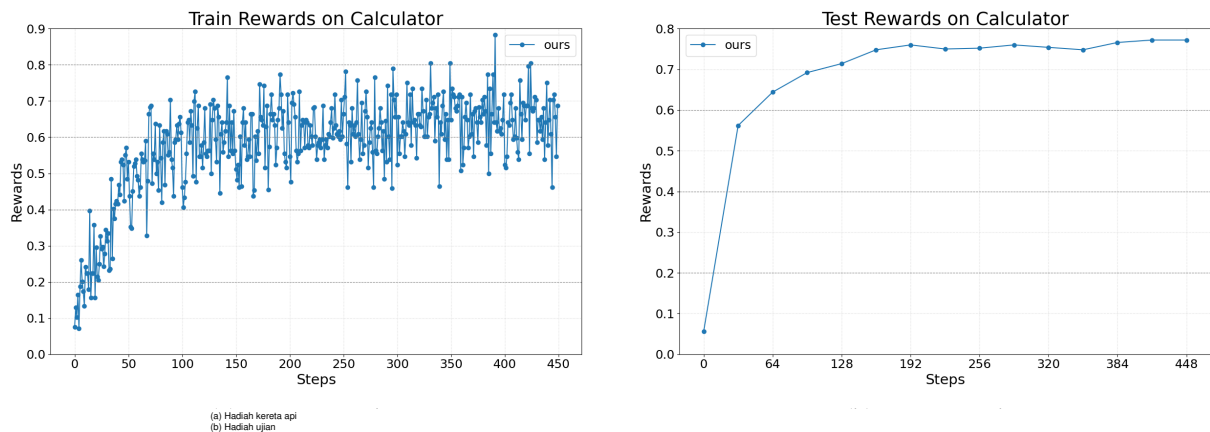
Seperti yang ditunjukkan pada Gambar 7, Agen Lightning meningkatkan kinerja secara konsisten selama pelatihan. Hal ini menunjukkan keefektifannya dalam pengaturan yang dilengkapi alat, di mana fungsi eksternal yang tepat membutuhkan dan berpikir keduanya diperlukan.

5 Diskusi

5.1 Pekerjaan Terkait

Kemajuan Terbaru dalam Multi-Turn RL untuk LLM

Dengan kemajuan RL untuk LLM dan kemunculannya kerangka pelatihan RL sumber terbuka seperti verl (Sheng et al., 2024), terdapat peningkatan minat terhadap memperluas kerangka kerja ini untuk mendukung skenario pelatihan agen yang lebih canggih. Karya terbaru, termasuk RAGEN (Wang et al., 2025b), Trinity-RFT (Pan et al., 2025), rLLM (Tan et al., 2025), SearchR1 (Jin et al., 2025; THU DM, 2025) dan lain-lain, telah menjelajahi RL dalam lingkungan interaktif multi-putaran.



Gambar 7: Kurva imbalan untuk tugas Kalkulator.

Metode ini biasanya menggabungkan agen berubah menjadi satu, urutan memanjang dan menugaskan spesifik masker untuk memastikan optimasi yang benar.

Alih-alih menggabungkan, kami memanfaatkan formulasi MDP, antarmuka data terpadu, dan penugasan kredit modul untuk mengatur data ke dalam transisi individual, yang menawarkan beberapa manfaat dalam menangkap kompleks karakteristik logika interaksi agen dunia nyata. Pertama, pemodelan berbasis transisi kami mendukung a berbagai arsitektur dan alur kerja agen (Anthropic, 2025), termasuk orkestrasi multi-agen. Fleksibilitas ini menjembatani kesenjangan antara pelatihan dan penerapan, memastikan pengoptimalan berbasis RL selaras dengan sifat dinamis dan beragam dari agen dunia nyata. Metode berbasis penggabungan adalah hanya cocok untuk sebagian kecil agen dengan alur kerja yang sederhana dan berurutan serta sulit ditangani pola yang lebih canggih. Kedua, dengan mengorganisasikan data sebagai transisi, bukan sebagai rangkaian, Pendekatan kami menghilangkan masalah panjang konteks akumulatif. Saat agen mengumpulkan konteks melalui interaksi multi-putaran, keluaran alat, pertukaran protokol (misalnya, MCP (Protokol, 2025)), dan multi-agen komunikasi, urutan yang dihasilkan sering kali melebihi batasan panjang input LLM atau kenaikan kebutuhan komputasi dan memori yang lebih tinggi untuk layanan pelatihan. Melihat transisi sebagai satu kesatuan sampel secara signifikan mengurangi peningkatan panjang urutan yang disebabkan oleh akumulasi belokan. Ketiga, milik kita Kerangka kerja ini menghilangkan kebutuhan akan strategi penyembunyian khusus karena hanya masukan LLM saat ini yang disertakan dalam sampel pelatihan. Padahal, untuk memastikan optimasi yang benar, penggabungan memerlukan desain strategi penyembunyian khusus untuk mekanisme masukan, kerugian, dan perhatian. Strategi penyembunyian ini adalah sering kali spesifik pada aplikasi, tidak dapat digeneralisasikan, dan memerlukan hubungan yang erat antara proses pelatihan dan logika eksekusi agen. Mereka juga mengganggu kontinuitas posisi, implementasi dan debugging lebih sulit. Keempat, formulasi berbasis transisi membuka potensi algoritma RL tingkat lanjut, seperti algoritma RL hierarki (Zhou et al., 2024), yang menawarkan penugasan kredit yang lebih efektif mekanisme. Dengan mengaktifkan fleksibilitas algoritmik, pendekatan kami membuka jalan bagi metode RL yang inovatif disesuaikan dengan skenario agen yang kompleks.

Sistem Pelatihan RL Skala Besar untuk LLM

Perkembangan RL untuk LLM didorong oleh stabil

dan sistem pelatihan yang kuat, termasuk verl (Sheng et al., 2024), OpenRLHF (Hu et al., 2024), TRL (von Werra dkk., 2020), ROLL (Wang dkk., 2025a), AReaL (Fu dkk., 2025), dan seterusnya. Sistem-sistem ini terutama fokus pada pelatihan LLM yang efisien dalam skenario putaran tunggal. Meskipun mereka juga menawarkan ekstensi untuk berbasis agen

aplikasi (Cao et al., 2025; verl, 2025), mereka umumnya mengharuskan pengembang untuk membangun kembali agen di dalam sistem pelatihan. Persyaratan ini mungkin timbul dari kenyataan bahwa kerangka kerja RL yang ada biasanya memerlukan hal tersebut sisi pelatihan untuk menyadari logika eksekusi agen untuk mengatur data pelatihan dengan benar, seperti menentukan urutan penggabungan dan di mana menerapkan masker. Akibatnya, agen seharusnya diimplementasikan dengan cara yang dipadukan erat dengan kerangka pelatihan. Namun, kendala ini adalah sulit dipenuhi mengingat keragaman ekosistem pengembangan agen yang ada. Dalam praktiknya, agen dunia nyata sangat dapat disesuaikan dan sering kali dikembangkan menggunakan berbagai kerangka kerja, seperti Agen OpenAI SDK (OpenAI, 2025), LangChain (LangChain, 2025), dan AutoGen (AutoGen, 2025), atau bahkan dibuat secara langsung dari awal dengan LLM SDK. Memigrasikan agen ini ke kerangka kerja RL yang ada biasanya memerlukan pengguna untuk mengadaptasi atau menerapkan kembali logika eksekusi agen secara manual agar sesuai dengan persyaratan kerangka kerja, yang merupakan proses padat karya, rawan kesalahan, dan sulit untuk diterapkan pada agen yang heterogen ekosistem. Selain itu, migrasi ini seringkali memerlukan keahlian khusus dalam kerangka pelatihan RL, seperti mengenal Ray (Moritz et al., 2018) atau sistem terdistribusi lainnya, yang dapat menghadirkan a penghalang yang signifikan bagi banyak pengembang. Selain itu, kopling yang ketat memberikan beban tambahan pada Proses pelatihan RL itu sendiri, karena agen mungkin melibatkan dependensi yang kompleks, termasuk berbagai server MCP,

alat eksternal, dan API. Mengintegrasikan komponen-komponen ini ke dalam kerangka pelatihan RL memperkenalkan kompleksitas dan overhead tambahan, sehingga mengurangi tujuan inti RL dan mempersulit pemeliharaan dan skalabilitas.

Kerangka kerja kami berbeda secara signifikan dari karya-karya ini dalam desain sistem, karena kami benar-benar memisahkan RL pelatihan dari agen. Hal ini menghilangkan kebutuhan untuk menulis logika pengumpulan data dalam mesin pelatihan RL dan hampir tidak memperkenalkan modifikasi kode pada sisi agen. Hasilnya, kami dapat berintegrasi dengan lancar dengan berbagai agen, baik yang dikembangkan menggunakan kerangka kerja yang berbeda atau dibuat dari awal pengguna. Selain itu, pemodelan kami secara teoritis mendukung integrasi dengan berbagai pelatihan RL kerangka kerja, yang selanjutnya meningkatkan fleksibilitas dan skalabilitas.

RL Multi-Putaran Berpusat pada Algoritma Berfungsi

RL multi-putaran telah menjadi topik yang menarik dalam algoritma RL komunitas, dengan karya seperti ArCher (Zhou et al., 2024) meneliti permainan teks dan WebShop (Yao et al., 2022) berfokus pada tugas e-commerce. Pekerjaan ini biasanya melibatkan agen RL yang berinteraksi dengan lingkungan dalam beberapa putaran, namun seringkali hanya menggunakan model kebijakan yang relatif kecil (misalnya, ukuran parameter lebih kecil dari 1B) atau Parameter Efficient Fine Tuning (PEFT) dalam eksperimen, sehingga sulit untuk digunakan kembali implementasinya dalam model skala besar.

Pelatihan RL Khusus Aplikasi

Sejak keberhasilan metode RL di DeepSeek-R1 (Guo et al., 2025), ada lonjakan minat untuk menerapkan RL dalam beberapa skenario tertentu, seperti RAG dan coding. Search-R1 (Jin et al., 2025) dan R1-Searcher (Song et al., 2025) menerapkan RL untuk melatih LLM agar menghasilkan lebih baik kueri dan meningkatkan kemampuan pencarian, mirip dengan contoh RAG kami di Bagian 4.2. DeepSWE (Luo dkk., 2025) menggunakan interaksi multi-putaran untuk mengajarkan LLM cara menulis kode, memanggil fungsi, dan menggunakan API. Di kami contoh text-to-SQL di Bagian 4.1, LLM menghasilkan kueri SQL sederhana. Sementara itu, beberapa berhasil seperti ReTool (Feng et al., 2025) dan SimpleTIR (Xue et al., 2025) bertujuan untuk meningkatkan integrasi alat jangka panjang penalaran dengan pembelajaran RL.

Pekerjaan-pekerjaan ini terutama berfokus pada tugas atau skenario tertentu, dan sering kali bergantung pada hal yang telah ditentukan sebelumnya alur kerja dalam pelatihan dan inferensi. Oleh karena itu, mereka tidak menargetkan pelatihan agen untuk tujuan umum. Di sebaliknya, pendekatan kami tidak terbatas pada tugas atau skenario tertentu; sebagai gantinya, kami menyediakan metode terpadu untuk agen AI mana pun, yang menjembatani agen AI dan metode pelatihan RL.

5.2 Pekerjaan Masa Depan

Metode Pengoptimalan Lainnya

Selain RL, kerangka pemodelan kami juga mendukung pengoptimalan lainnya metode juga, seperti optimasi cepat otomatis yang disebutkan di Bagian 3.1.3. Secara lebih umum berbicara, fokus pada komponen utama dan pemanggilannya adalah cara utama untuk optimalisasi agen, tidak terbatas pada metode berbasis RL. Oleh karena itu, kami memperkenalkan konsep Component of Interest (Col) kepada tentukan subset komponen dalam jejak eksekusi yang harus dioptimalkan. Misalnya, rendering templat yang cepat dapat diperlakukan sebagai panggilan alat, dan dengan memperlakukan alat ini sebagai Col, Agen Lightning dapat memfasilitasi metode pengoptimalan yang cepat. Struktur data yang terpadu dan dapat diperluas ini mendukung optimasi hilir yang komprehensif dan analisis perilaku agen. Memperluas dukungan ke lebih banyak lagi metode pengoptimalan adalah bagian dari peta jalan pengembangan kami di masa depan.

Perbaikan Algoritma RL

Mengembangkan algoritma RL yang lebih efisien adalah kunci untuk menyelesaikan model optimasi dalam skenario agen yang lebih kompleks. Aspek-aspek ini termasuk namun tidak terbatas pada jangka panjang penugasan kredit, algoritma eksplorasi, algoritma di luar kebijakan, dan sebagainya. Model Agen Lightning dan mengatur data dalam hal transisi, membuatnya lebih mudah untuk mengintegrasikan algoritma tambahan.

Kemajuan Infrastruktur Sistem RL untuk Agen

Infrastruktur yang mendukung RL untuk LLM terus berkembang, menghadirkan peluang signifikan untuk pengembangan bersama dengan kerangka kerja RL agen seperti Agent Lightning. Salah satu arah yang menjanjikan adalah pemilihan lebih lanjut komponen sistem—khususnya, memisahkan alur kerja pelatih, peluncuran (mesin inferensi), dan agen—untuk mengatasi permasalahan yang ada. Hambatan peluncuran dan meningkatkan skalabilitas dalam pelatihan RL skala besar. Menjelajahi peningkatan arsitektur seperti itu dapat menghasilkan pipeline RL yang lebih efisien dan fleksibel. Selain itu, optimalisasi untuk jangka panjang tugas akan mendapat manfaat dari inovasi terkoordinasi dalam algoritma RL dan desain sistem, sehingga memungkinkan lebih banyak hal pelatihan efektif agen kompleks.

Penyajian yang Efisien

Untuk penyajian LLM, pendekatan tingkat lanjut dengan abstraksi yang lebih ramah LLM (misalnya, Parrot (Lin et al., 2024)) dapat dieksplorasi untuk mengoptimalkan pemanfaatan sumber daya dan waktu respons. Untuk konteks yang panjang akselerasi, teknik seperti komputasi renggang untuk pemrosesan konteks panjang (misalnya, Minference (Jiang dkk., 2024)) dapat meningkatkan kinerja secara signifikan. Selain itu, penjadwalan sumber daya yang lebih baik untuk penayangan lingkungan dan alat dapat lebih menyederhanakan operasi dan meningkatkan skalabilitas dalam penerapan yang beragam skenario.

Referensi

Mart'in Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, dkk. {TensorFlow}: sistem untuk {Skala Besar} pembelajaran mesin. Dalam simposium USENIX ke-12 tentang desain dan implementasi sistem operasi (OSDI 16), hal.265–283, 2016.

AgenOps. AgenOps. <https://www.agentops.ai/>, 2025.

AI Tembakan Bulan.
Peneliti Kimi:
Pelatihan RL menyeluruh untuk kemampuan agen yang muncul.
<https://moonshotai.github.io/Kimi-Researcher/>, Juni 2025.

V Aho Alfred, S Lam Monica, dan D Ullman Jeffrey. Prinsip, teknik & alat penyusun. pirson Pendidikan, 2007.

Anthropic. Membangun agen AI yang efektif. <https://www.anthropic.com/engineering/building-Effectiveagents>, Desember 2024.

Anthropic. Bagaimana kami membangun sistem penelitian multi-agen kami. <https://www.anthropic.com/engineering/builtmulti-agent-research-system>, Juni 2025.

Generasi Otomatis. Generasi Otomatis. <https://github.com/microsoft/autogen>, 2025.

TELUK. Bge-besar-en-v1.5. <https://huggingface.co/BAAI/bge-large-en-v1.5>, 2023.

Shiyi Cao, Sumanth Hegde, Dacheng Li, Tyler Griggs, Shu Liu, Eric Tang, Jiayi Pan, Xingyao Wang, Akshay Malik, Graham Neubig, Kourosh Hakhmaneshi, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, dan Ion Stoica. Skyrl-v0: Latih agen cakrawala panjang dunia nyata melalui penguatan pembelajaran, 2025.

Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, dan Shunyu Yao. Tindakan Kebakaran: Menuju Penyempurnaan Agen Bahasa, Oktober 2023.

Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, dan Wanjun Zhong. Retool: Pembelajaran penguatan untuk penggunaan alat strategis di bidang ilmu pengetahuan. pracetak arXiv arXiv:2504.11536, 2025.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, dan Yi Wu. Areal: Penguatan asinkron skala besar sistem pembelajaran penalaran bahasa, 2025. URL <https://arxiv.org/abs/2505.24298>.

Daya Guo, Dejian Yang, Haowei Zhang, Lagu Junxiao, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, dkk. Deepseek-r1: Memberi insentif pada kemampuan penalaran dalam ilmu pengetahuan melalui penguatan belajar. arXiv pracetak arXiv:2501.12948, 2025.

Jian Hu. Reinforce++: Pendekatan sederhana dan efisien untuk menyelaraskan model bahasa besar. pracetak arXiv arXiv:2501.03262, 2025.

Jian Hu, Xibin Wu, Zilin Zhu, Weixun Wang, Dehao Zhang, Yu Cao, dkk. Openrlhf: Mudah digunakan, kerangka kerja rlhf yang skalabel dan berkinerja tinggi. arXiv pracetak arXiv:2405.11143, 2024.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, dkk. Minference 1.0: Mempercepat pengisian awal untuk pendidikan konteks panjang melalui perhatian dinamis yang jarang. Kemajuan dalam Sistem Pemrosesan Informasi Neural, 37: 52481–52515, 2024.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, dan Jiawei Han. Search-r1: Pelatihan untuk berpikir dan memanfaatkan mesin pencari dengan pembelajaran penguatan. arXiv pracetak arXiv:2503.09516, 2025.

Marek Kadl'cík, Michal Štef'ánek, Ondřej Sotol'a'r, dan Vlastimil Martinek. Calc-x dan calcformers: Memberdayakan rantai pemikiran aritmatika melalui interaksi dengan sistem simbolik. Dalam Prosiding The Konferensi 2023 tentang Metode Empiris dalam Pemrosesan Bahasa Alami: Jalur utama, Singapura, Singapura, Desember 2023. Asosiasi Linguistik Komputasi. URL <https://arxiv.org/abs/2305.15017>.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, dan Ion Stoica. Manajemen memori yang efisien untuk penyajian model bahasa yang besar dengan perhatian halaman. Dalam Prosiding Simposium ACM SIGOPS ke-29 tentang Prinsip Sistem Operasi, 2023.

LangChain. LangChain. <https://www.langchain.com/>, 2025.

Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, dan Lili Qiu. Parrot: Penyajian aplikasi berbasis llm yang efisien dengan variabel semantik. Dalam Simposium USENIX ke-18 tentang Desain dan Implementasi Sistem Operasi (OSDI 24), Santa Clara, CA, Juli 2024. Asosiasi USENIX. URL <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>.

Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, dan Yiling Lou. Agen Berbasis Model Bahasa Besar untuk Rekayasa Perangkat Lunak: Sebuah Survei. <https://arxiv.org/abs/2409.02977v1>, September 2024.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, dan Igor Mordatch. Multi-agen aktor-kritikus untuk lingkungan campuran kooperatif-kompetitif. Kemajuan dalam pemrosesan informasi saraf sistem, 30, 2017.

Michael Luo, Interminded Jaskirat Singh, Siskirat Singh, Ameen Patel, Of Wukayan Wu, Clini Kai, Trun Venat, Shang Zhu, Bethirathwan, Water, Ce Zhang, Li Errra Li, Raluca Ada Paus, Kushik Sen, dan Ion Styica. Deeps: Melatih pengkodean peristiwa yang canggih dari scram oleh akademisi rl. <https://preptty-ab7> Sel-Ageth-Ageth-Scaling-22228190219333030300. 2025. Gagasan blog.

Ankit Maloo. Pelajaran Pahit: Memikirkan Kembali Cara Kita Membangun Sistem AI. <https://ankitmaloo.com/bitterlesson/>, Maret 2025.

AI Meta. Llama 3.2: Merevolusi edge ai dan vision dengan model terbuka dan dapat disesuaikan. Blog Meta AI. Diakses pada Desember, 20:2024, 2024.

MoonshotAI. Kimi k2: Kecerdasan agen terbuka. <https://moonshotai.github.io/Kimi-K2/>, 2025.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexei Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, dan Ion Stoica. Ray: kerangka terdistribusi untuk aplikasi ai yang sedang berkembang. Dalam Prosiding Konferensi USENIX ke-13 tentang Desain Sistem Operasi dan Implementasinya, OSDI'18, hlm. 561–577, AS, 2018. Asosiasi USENIX. ISBN 9781931971478.

OpenAI. SDK agen Openai. <https://openai.github.io/openai-agents-python/>, 2025.

OpenTelemetry. OpenTelemetry. <https://opentelemetry.io/>, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, dkk. Melatih model bahasa untuk mengikuti instruksi dengan umpan balik manusia. Kemajuan dalam sistem pemrosesan informasi saraf, 35:27730–27744, 2022.

Xuchen Pan, Yanxi Chen, Yushuo Chen, Yuchang Sun, Daoyuan Chen, Wenhao Zhang, Yuexiang Xie, Yilun Huang, Yilei Zhang, Dawei Gao, dkk. Trinity-rft: Kerangka kerja dengan tujuan umum dan terpadu untuk penguatan penyempurnaan model bahasa besar. arXiv pracetak arXiv:2505.17826, 2025.

Protokol Konteks Model. Protokol Konteks Model (MCP). <https://modelcontextprotocol.io/introduction>, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, dkk. Deepseekmath: Mendorong batas penalaran matematis secara terbuka model bahasa. arXiv pracetak arXiv:2402.03300, 2024.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, dan Yueting Zhuang. MemelukGPT: Menyelesaikan tugas AI dengan ChatGPT dan teman-temannya secara berkelompok, Desember 2023.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, dan Chuan Wu. Hybridflow: Kerangka kerja rlhf yang fleksibel dan efisien. arXiv pracetak arXiv: 2409.19256, 2024.

Gravitasi Signifikan. OtomatisGPT, 2023.

David Silver dan Richard S Sutton. Selamat datang di era pengalaman. Google AI, 1 Agustus 2025.

Lagu Huatong, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, dan Ji-Rong Wen. R1-Searcher: Memberi Insentif pada Kemampuan Pencarian di LLM melalui Pembelajaran Penguatan, Maret 2025.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, dan Yunfeng Liu. Roformer: Ditingkatkan transformator dengan penyematan posisi putar. *Komputasi saraf*, 568:127063, 2024.

Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnav Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, dan Ion Stoica. rmlm: A kerangka kerja untuk agen bahasa pasca pelatihan. <https://pretty-radio-b75.notion.site/rLLM-A-F-ramework-untuk-Pasca-Pelatihan-Bahasa-Agen-21b81902c146819db63cd98a54ba5f31>, 2025. Gagasan blog.

Tim Kimi, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, dkk. Kimi k1.5: Menskalakan pembelajaran penguatan dengan llms. *arXiv preprint arXiv:2501.12599*, 2025.

Tim SGLang. SGLang. <https://github.com/sgl-project/sglang>, 2025.

THUDM. lendir. <https://github.com/THUDM/slime>, 2025.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, dan Ashish Sabharwal. MuSiQue: Pertanyaan multihop melalui komposisi pertanyaan single-hop. *Transaksi Asosiasi Komputasi Linguistik*, 2022.

agen cuti ringan berjalan. [https://verl.readthedocs.io/en/latest/advance/agent loop.html](https://verl.readthedocs.io/en/latest/advance/agent%20loop.html), 2025.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, dan Quentin Gallou  dec. Trl: Pembelajaran penguatan transformator. <https://github.com/huggingface/trl>, 2020.

Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Li Zhendong, Li Xiaoyang, dkk. Optimalisasi pembelajaran penguatan untuk pembelajaran skala besar: An perpustakaan penskalaan yang efisien dan ramah pengguna. *arXiv preprint arXiv:2506.06122*, 2025a.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, dkk. Ragen: Memahami evolusi diri pada agen llm melalui pembelajaran penguatan multi-putaran. *arXiv preprint arXiv:2504.20073*, 2025b.

Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, dan Chong Luo. Logic-rl: Melepaskan penalaran ilmiah dengan pembelajaran penguatan berbasis aturan. *arXiv preprint arXiv:2502.14768*, 2025.

Renjun Xu dan Jingwen Peng. Survei Komprehensif Penelitian Mendalam: Sistem, Metodologi, dan Lamaran, Juni 2025.

Zhenghai Xue, Longtao Zheng, Qian Liu, Yingru Li, Zejun Ma, dan Bo An. Simpletir : Ujung ke ujung pembelajaran penguatan untuk penalaran terintegrasi alat multi-putaran. <https://simpletir.notion.site/re-pelabuhan>, 2025. Blog Gagasan.

Shunyu Yao. Babak Kedua. <https://ysymyth.github.io/The-Second-Half/>, 2025.

Shunyu Yao, Howard Chen, John Yang, dan Karthik Narasimhan. Webshop: Menuju dunia nyata yang terukur interaksi web dengan agen bahasa yang membumi. *Kemajuan dalam Sistem Pemrosesan Informasi Neural*, 35: 20744–20757, 2022.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, dkk. Spider: Kumpulan data berlabel manusia berskala besar untuk kompleks dan lintas domain penguraian semantik dan tugas text-to-sql. Dalam *Prosiding Konferensi Metode Empiris 2018 di Pemrosesan Bahasa Alami*, hlm. 3911–3921, 2018.

Kaiqing Zhang, Zhuoran Yang, dan Tamer Basar. Pembelajaran penguatan multi-agen: Selektif gambaran umum teori dan algoritma. *Buku Pegangan pembelajaran dan kontrol penguatan*, hlm. 321–384, 2021.

Peitian Zhang, Shitao Xiao, Zheng Liu, Zhicheng Dou, dan Jian-Yun Nie. Ambil apa pun untuk ditambah model bahasa besar. *arXiv preprint arXiv:2310.07554*, 2023.

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, dan Aviral Kumar. Pemanah: Bahasa pelatihan agen model melalui rl multi-putaran hierarki. Dalam *Konferensi Internasional Keempat Puluh Satu tentang Pembelajaran Mesin*, 2024.

SEBUAH
Contoh Mengoptimalkan Agen yang Ada dengan Agen Lightning

Berikut contoh cara mengoptimalkan agen yang ada menggunakan Agent Lightning. Diberikan agen sederhana yang berinteraksi dengan lingkungan permainan (misalnya 20 pertanyaan atau tebakan angka), kita dapat menggunakan Kerangka kerja Agen Lightning untuk mengoptimalkan kinerjanya tanpa mengubah kode agen itu sendiri. Ini dia struktur folder khas untuk agen:

```
agen/  
|- data/train.jsonl  
|- lingkungan/game_server.py  
|- agen.py
```

Listing 1: Struktur Folder Kode Agen

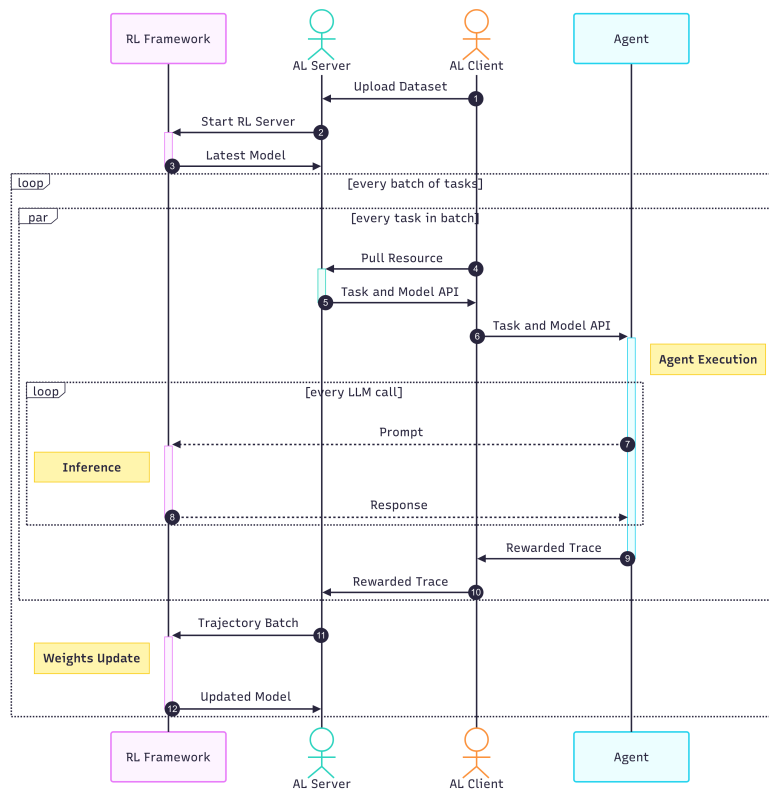
Dalam contoh ini, file `agent.py` berisi fungsi logika agen `agent function`, yang menerima titik akhir LLM dan lingkungan `game`, dan mengembalikan jawaban ke `game`. Server permainannya adalah dibuat oleh `environment/game_server.py` dengan pasangan (`game seed`, `ground truth`) yang disimpan di `data/train.jsonl`.

Untuk mengoptimalkan agen ini menggunakan Agent Lightning, kita dapat membuat file `train.py` yang menggunakan Agent Lightning kerangka kerja untuk mengoptimalkan kinerja agen tanpa mengubah kode agen. Ini sebuah contoh tentang bagaimana melakukan ini4:

```
1 # kereta.py  
2  
3 dari agen impor agent_function  
4 dari lingkungan.game_server impor GameServer  
5 dari agent_lightning impor Klien, Sumber Daya, Tugas  
6  
7 # TODO: lingkungan dapat dikelola secara terpisah  
8 permainan_server = PermainanServer()  
9  
10 # Tugas sesuai dengan objek data yang disimpan dalam file "jsonl"  
11 # Sumber daya adalah meta data yang diambil dari server,  
12  
13 # dan berisi titik akhir LLM  
14 def agent_run(sumber_data: Sumber daya, tugas: Tugas):  
15     game_url = game_server.create_game(tugas.game_seed)  
16     jawaban = agen_fungsi(resource.model_api, game_url)  
17     hadiah = game_url.score(jawaban, tugas.ground_truth)  
18     imbalan kembali  
19  
20 # mulai AgentLightningServer dan teruskan URL-nya ke dalam variabel lingkungan  
21 klien = Klien(os.environ["AgentLightningServerUrl"])  
22 # Anda juga dapat menentukan id data yang telah diunggah sebelumnya  
23 klien.upload_data("data/train.jsonl", test_file=Tidak Ada)  
24 klien.train(agent_run, nworkers=1)
```

Daftar 2: Skrip Pelatihan Agen Lightning

4API sumber terbuka mungkin mengalami modifikasi. Silakan merujuk ke repositori GitHub untuk informasi terbaru informasi.



Gambar 8: Diagram proses